

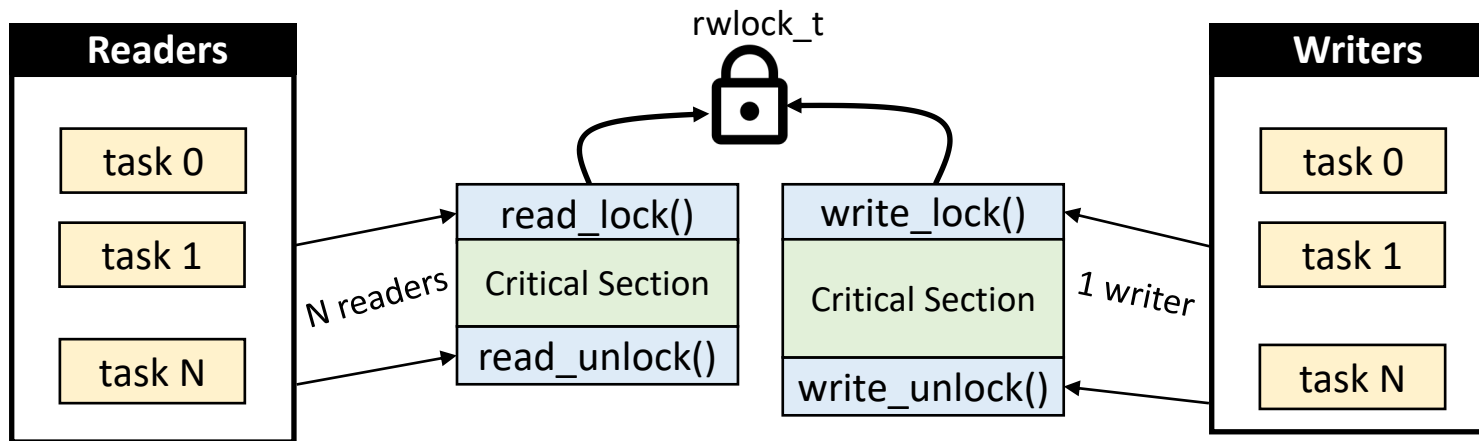
Linux Synchronization Mechanism: RCU (Read-Copy-Update)

Adrian Huang | Apr, 2023

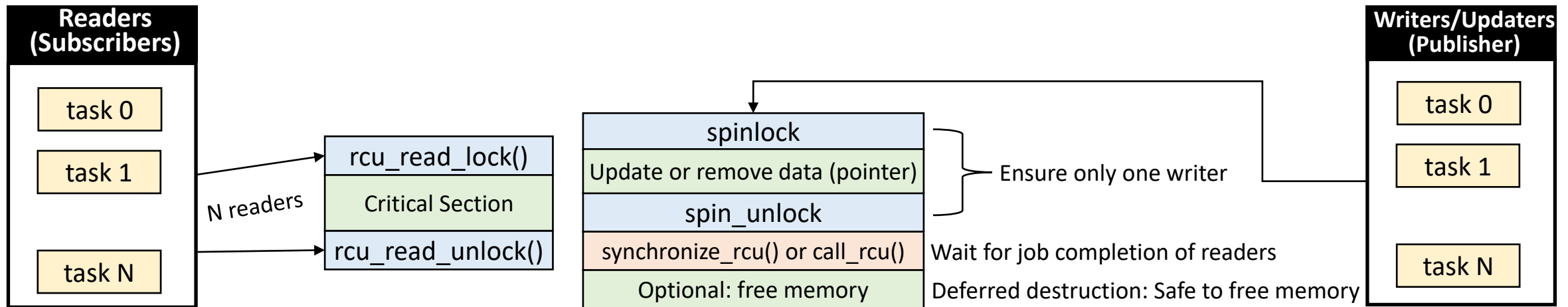
Agenda

- [Overview] rwlock (reader-writer spinlock) vs RCU
- RCU Implementation Overview
 - ✓ High-level overview
 - ✓ RCU: List Manipulation – Old and New data
 - ✓ Reader/Writer synchronization: six basic APIs
 - Reader
 - rcu_read_lock() & rcu_read_unlock()
 - rcu_dereference()
 - Writer
 - rcu_assign_pointer()
 - synchronize_rcu() & call_rcu()
- Classic RCU vs Tree RCU
- RCU Flavors
- RCU Usage Summary & RCU Case Study

[Overview] rwlock (reader-writer spinlock) vs RCU

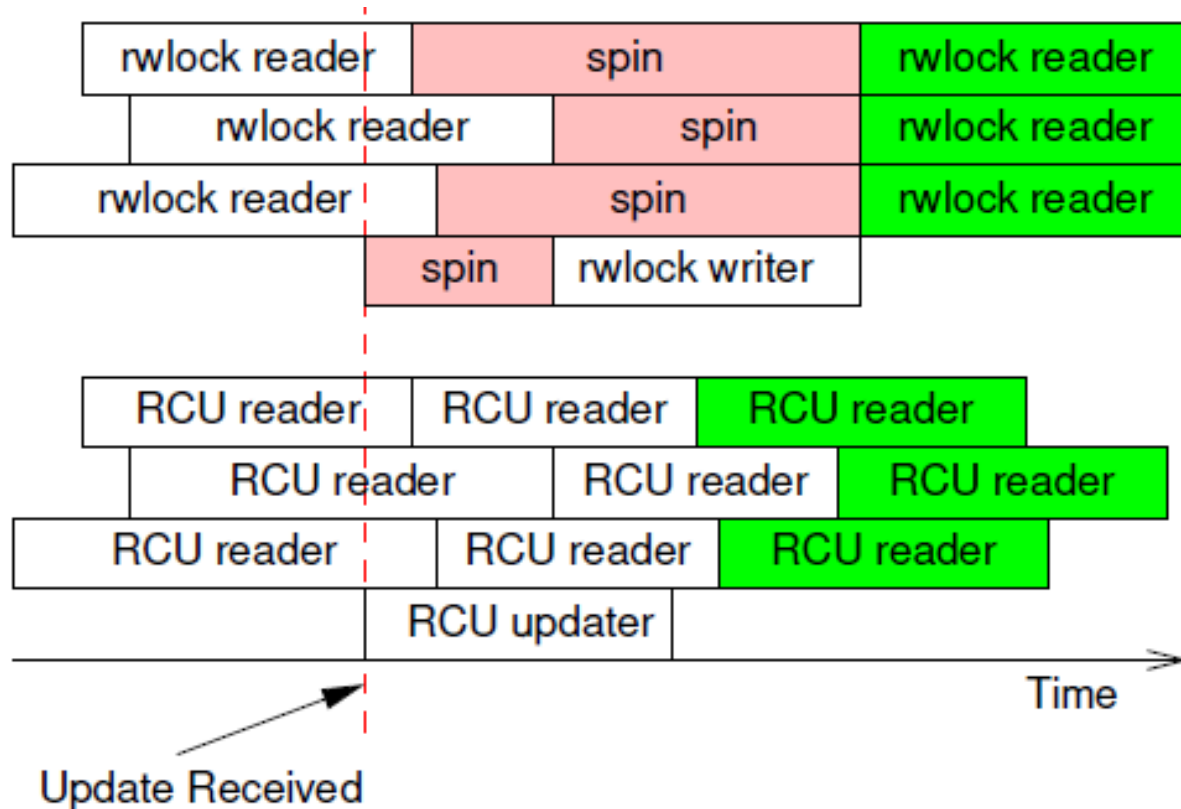


1. Mutual exclusion between reader and writer
2. Writer might be starved



1. RCU is a non-blocking synchronization mechanism: No mutual exclusion between readers and a writer
2. No specific *lock* data structure

[Overview] rwlock (reader-writer spinlock) vs RCU



rwlock

- Mutual exclusion between reader and writer
- Writer might be starved

RCU

- A non-blocking synchronization mechanism
- No specific *lock* data structure

* Reference from section 9.5 of [Is Parallel Programming Hard, And, If So, What Can You Do About It?](#)

**How does RCU achieve concurrent readers and one writer?
(No mutual exclusion between readers and one writer)**

Agenda

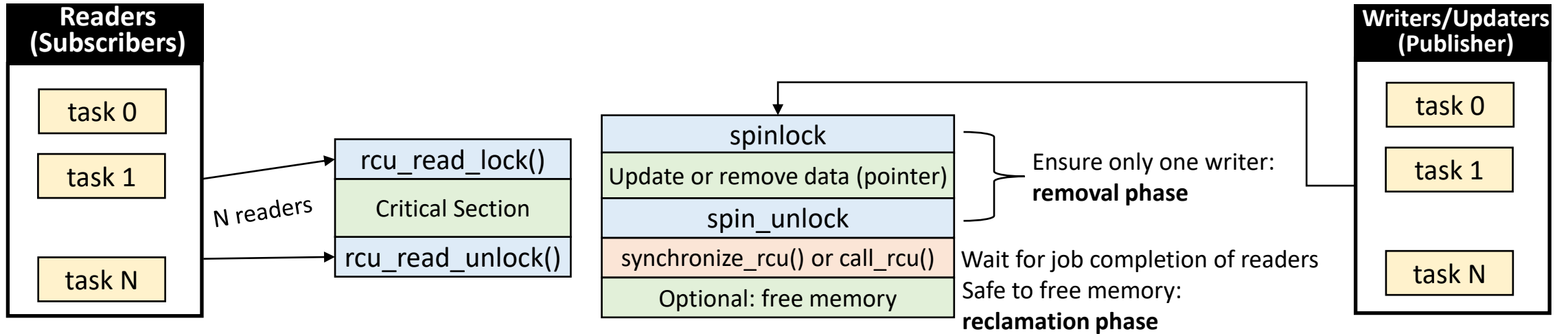
- [Overview] rwlock (reader-writer spinlock) vs RCU
- RCU Implementation Overview
 - ✓ High-level overview
 - ✓ RCU: List Manipulation – Old and New data
 - ✓ Reader/Writer synchronization: six basic APIs
 - Reader
 - rcu_read_lock() & rcu_read_unlock()
 - rcu_dereference()
 - Writer
 - rcu_assign_pointer()
 - synchronize_rcu() & call_rcu()
- Classic RCU vs Tree RCU
- RCU Flavors
- RCU Usage Summary & RCU Case Study

6 important APIs

Readers or Updaters	Primitive	Purpose
Readers	<code>rcu_read_lock()</code>	Start an RCU read-side critical section
	<code>rcu_read_unlock()</code>	End an RCU read-side critical section
	<code>rcu_dereference()</code>	Safely load an RCU-protected pointer
Updaters	<code>synchronize_rcu()</code>	Wait for all pre-existing RCU read-side critical sections to complete
	<code>call_rcu()</code>	Invoke the specified function after all pre-existing RCU read-side critical sections complete
	<code>rcu_assign_pointer()</code>	Safely update an RCU-protected pointer

RCU manipulates pointers

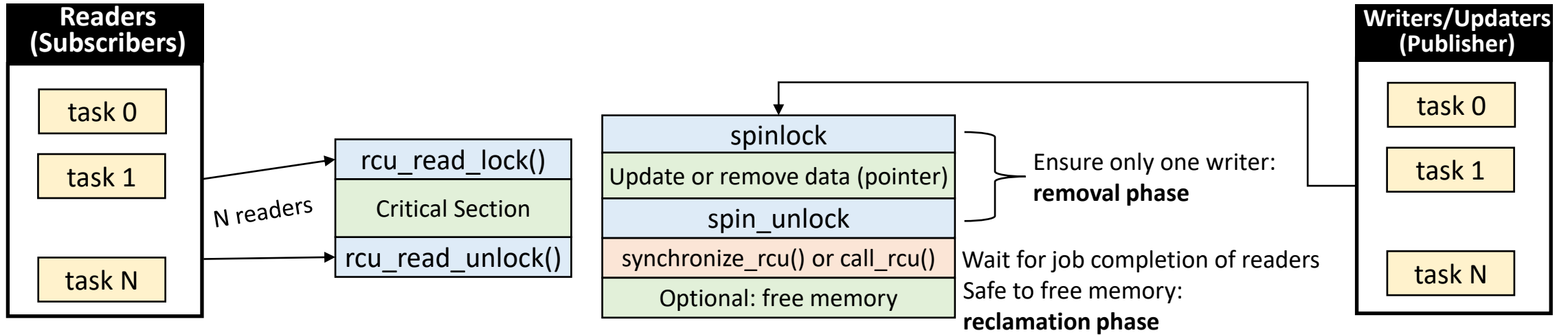
RCU Implementation Overview



Reader

- RCU is often used as a replacement for reader-writer locking
- Lockless readers
 - ✓ `rcu_read_lock()` simply disables preemption: No lock mechanism (Ex: spinlock, mutex and so on)
 - ✓ Readers need not wait for updates (non-blocking): low overhead and excellent scalability

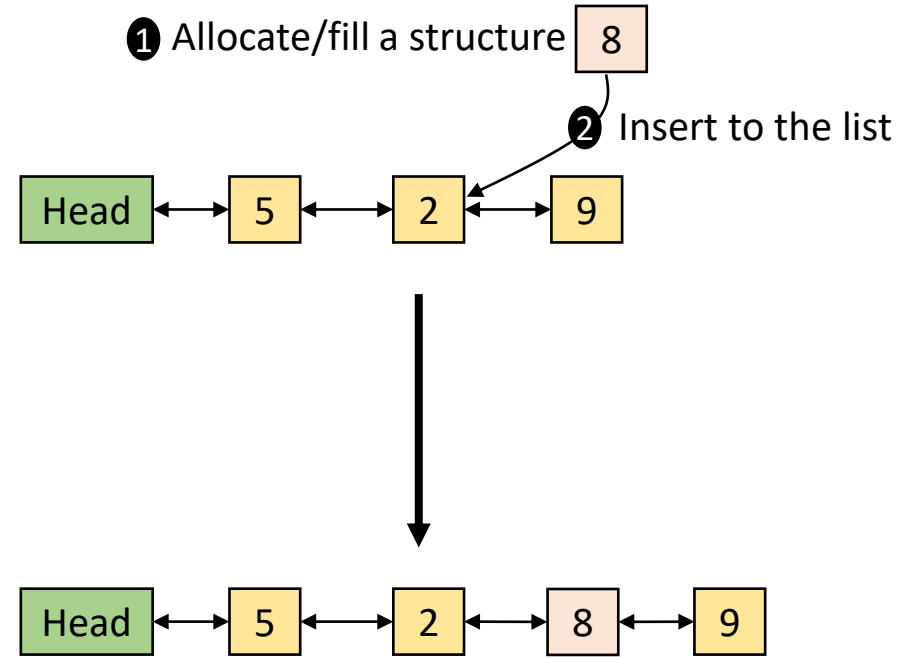
RCU Implementation Overview



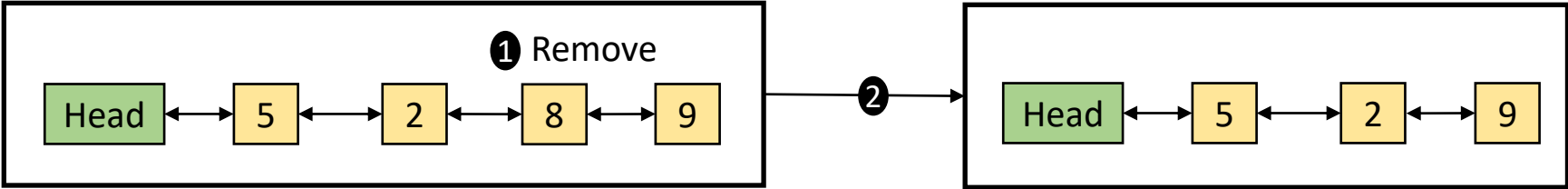
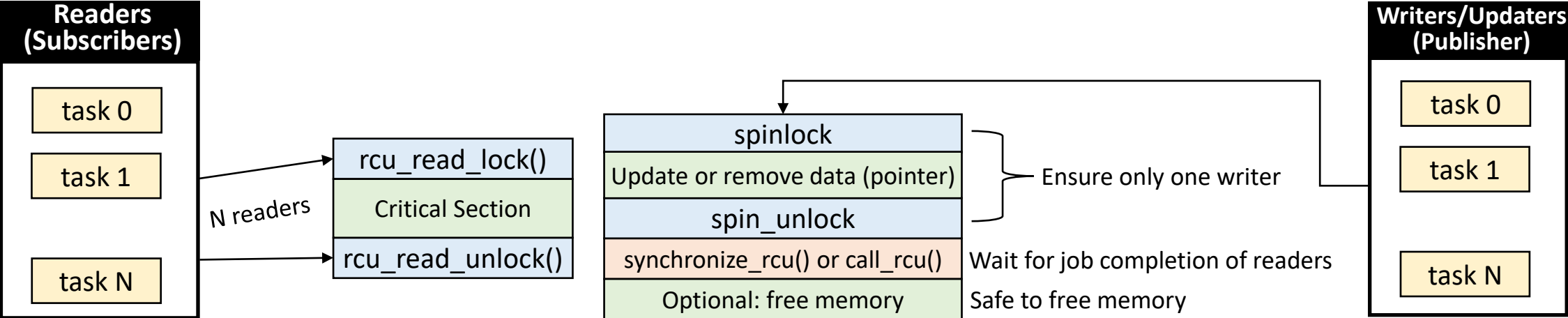
Typical RCU Update Sequence

- [Writer] Waiting for readers
 - ✓ Removal phase: remove references to data items (possibly by replacing them with references to new versions of these data items)
 - Can run concurrently with readers: readers see either the old or the new version of the data structure rather than a partially updated reference.
 - ✓ `synchronize_rcu()` and `call_rcu()`: wait for readers exiting critical section
 - ✓ Block or register a callback that is invoked after active readers have completed.
 - ✓ Reclamation phase: reclaim data items.
- Quiescent State (QS): [per-core] The time after pre-existing readers are done
 - ✓ Context switch: a valid quiescent state
- Grace Period (GP): All cores have passed through the quiescent state → Complete a grace period

RCU: List Manipulation – Old and New data



RCU: List Manipulation – Old and New data

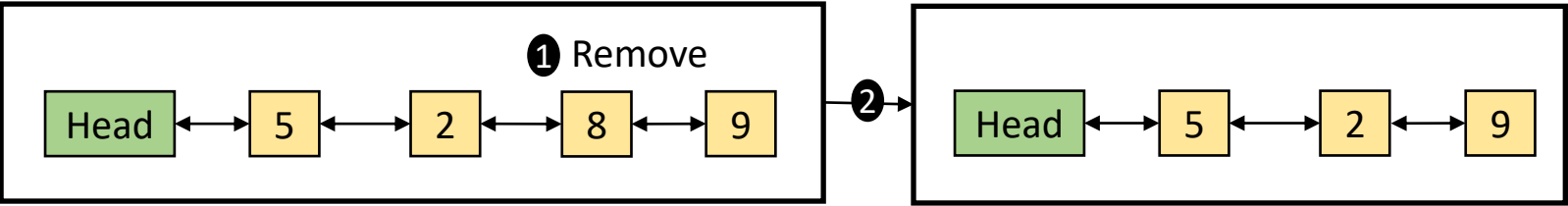


Old data 2 ↔ 8

New data 2 ↔ 9

Either the old data or the new data is read by readers.

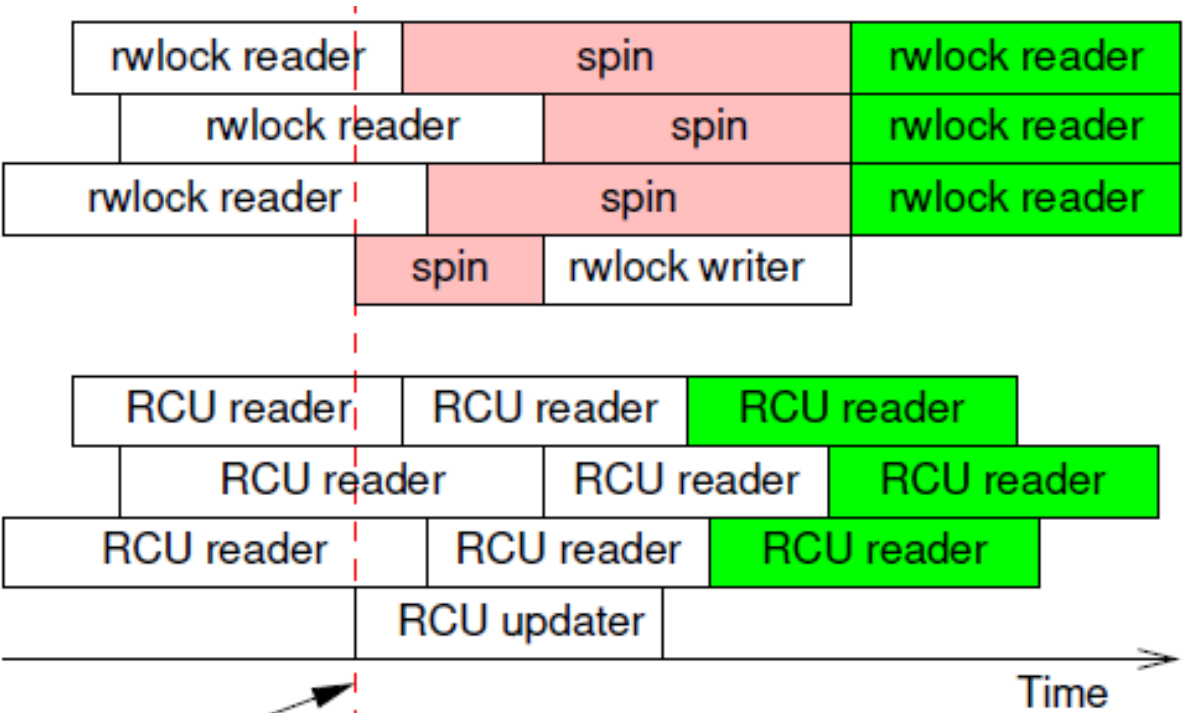
RCU: List Manipulation – Old and New data



Old data 2 ↔ 8

New data 2 ↔ 9

Either the old data or the new data is read by readers.



Legend

RCU reader

 Old data is read

RCU reader

 New data is read

Update Received

* Reference from section 9.5 of [Is Parallel Programming Hard, And, If So, What Can You Do About It?](#)

Reader/Writer synchronization

RCU Reader

<code>rcu_read_lock()</code>
<code>ptr = rcu_dereference(shared_abc);</code>
<code>printk("%d\n", ptr->number);</code>
<code>rcu_read_unlock()</code>

RCU Writer w/ valid pointer assignment

<code>spinlock</code>
<code>rcu_assign_pointer(shared_abc, ptr);</code>
<code>spin_unlock</code>
<code>synchronize_rcu()</code>
Optional: free memory

Ensure only one writer

Wait for job completion of readers

Safe to free memory

RCU Writer w/ NULL pointer assignment and free memory

<code>spinlock</code>
<code>rcu_assign_pointer(shared_abc, NULL);</code>
<code>spin_unlock</code>
<code>synchronize_rcu();</code>
<code>kfree(tmp);</code>

RCU Spatial/Temporal Synchronization: Sample Code

RCU Reader

```
struct myconfig {
    int a;
    int b;
} *curconfig;

int get_config(int *cur_a, int *cur_b)
{
    struct myconfig *mcp;

    rcu_read_lock();
    mcp = rcu_dereference(curconfig);
    if (!mcp) {
        rcu_read_unlock();
        return 0;
    }

    *cur_a = mcp->a;
    *cur_b = mcp->b;
    rcu_read_unlock();

    return 1;
}
```

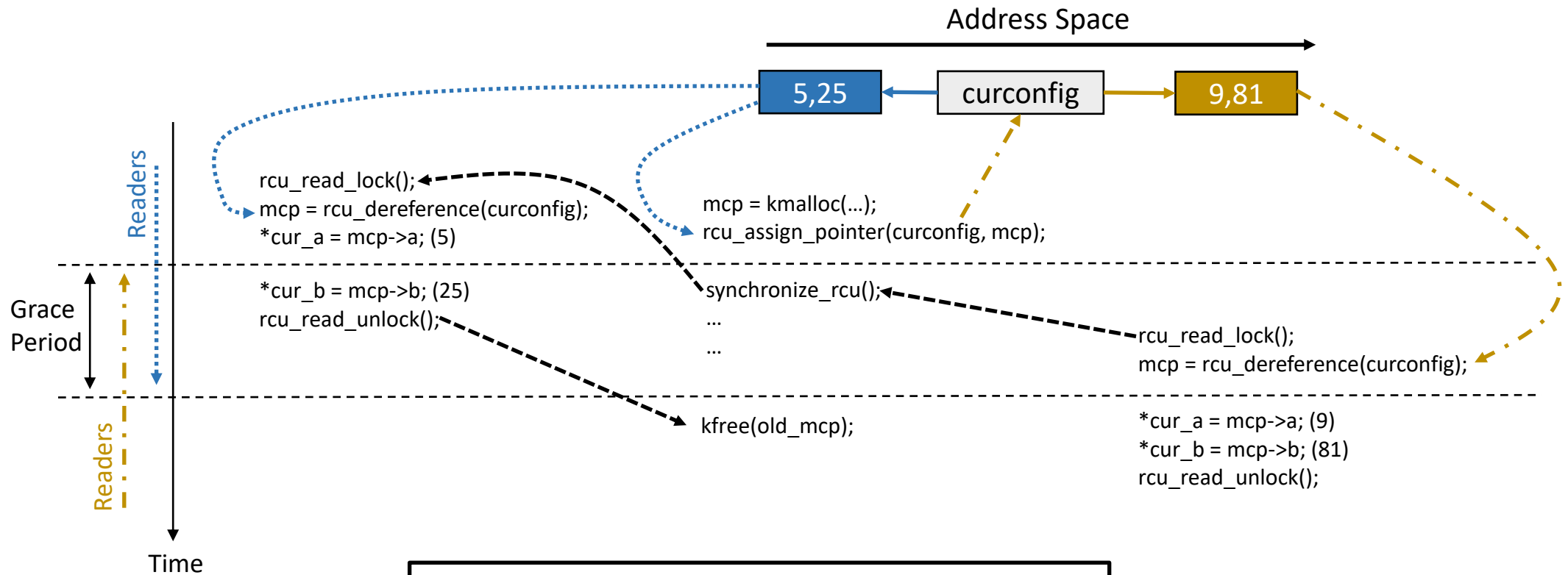
RCU Writer/Updater

```
void set_config(int cur_a, int cur_b)
{
    struct myconfig *old_mcp = rcu_dereference(curconfig);
    struct myconfig *mcp;

    mcp = kmalloc(sizeof(*mcp));
    BUG_ON(!mcp);
    mcp->a = cur_a;
    mcp->b = cur_b;
    rcu_assign_pointer(curconfig, mcp);

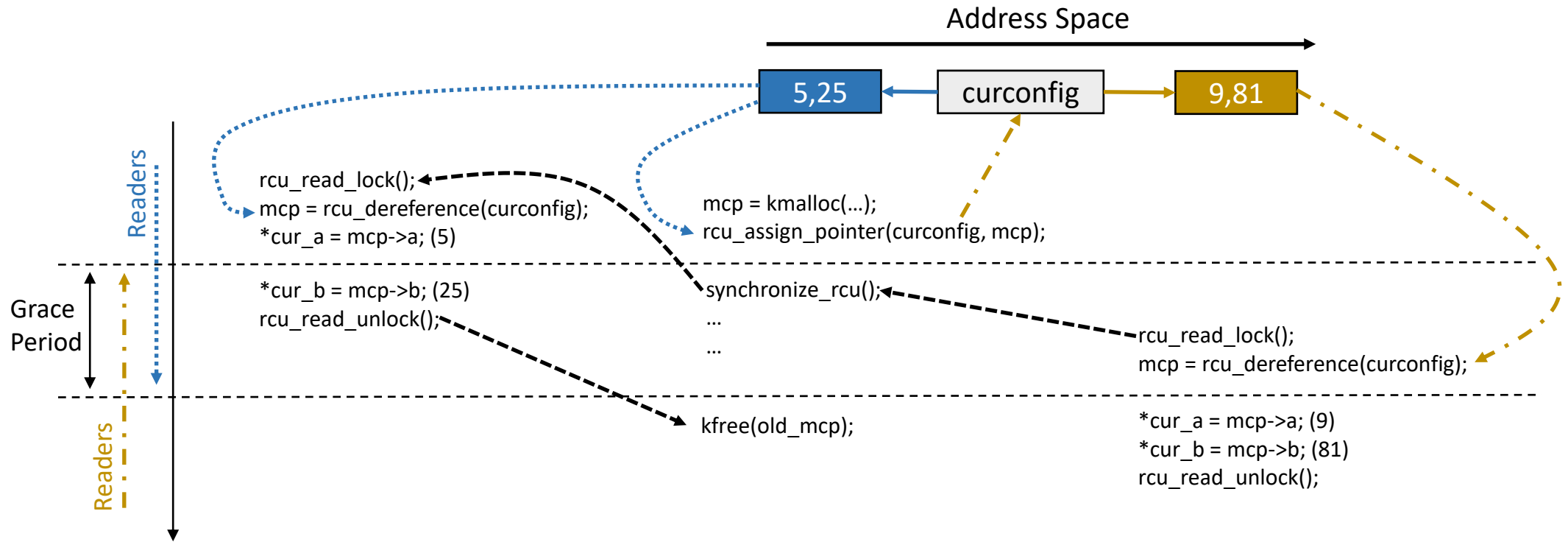
    if (old_mcp) {
        synchronize_rcu();
        kfree(old_mcp);
    }
}
```

RCU Spatial/Temporal Synchronization



- Temporal Synchronization
 - ✓ [Reader] `rcu_read_lock()` / `rcu_read_unlock()`
 - ✓ [Writer/Update] `synchronize_rcu()` / `call_rcu()`
- Spatial Synchronization
 - [Reader] `rcu_dereference()`
 - [Writer/Update] `rcu_assign_pointer()`

RCU Spatial/Temporal Synchronization

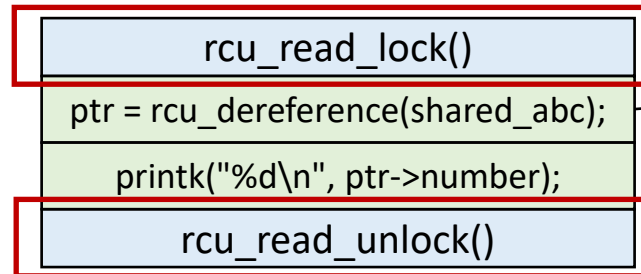


- Temporal Synchronization
 - ✓ [Reader] `rcu_read_lock()` / `rcu_read_unlock()`
 - ✓ [Writer/Update] `synchronize_rcu()` / `call_rcu()`
- Spatial Synchronization
 - [Reader] `rcu_dereference()`
 - [Writer/Update] `rcu_assign_pointer()`

RCU combines temporal and spatial synchronization in order to approximate reader-writer locking

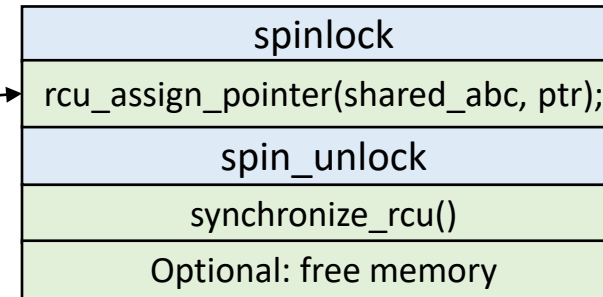
Reader: rcu_read_lock() & rcu_read_unlock():

RCU Reader



Is new data or old data read?

RCU Writer w/ valid pointer assignment



Ensure only one writer

Wait for job completion of readers

Safe to free memory

```
#define rcu_read_lock() \
do { \
    preempt_disable(); \
    __acquire(RCU); \
    rcu_read_acquire(); \
} while(0)
```

`include/linux/rcupdate.h`

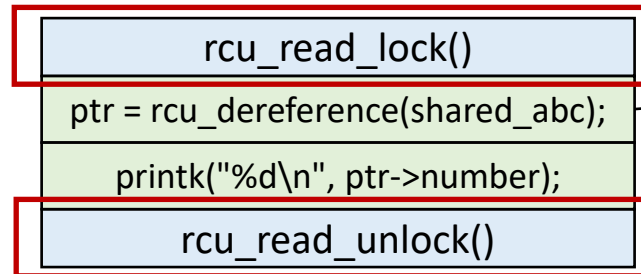
```
#define rcu_read_unlock() \
do { \
    rcu_read_release(); \
    __release(RCU); \
    preempt_enable(); \
} while(0)
```

`include/linux/rcupdate.h`

- ✓ Simply disable/enable preemption when entering/exiting RCU critical section
 - [Why] A QS is detected by a context switch.

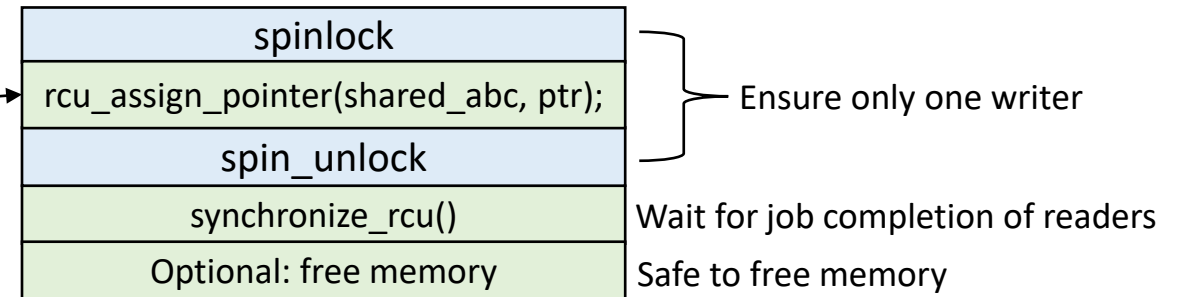
Reader: rcu_read_lock() & rcu_read_unlock()

RCU Reader



Is new data or old data read?

RCU Writer w/ valid pointer assignment



```

#define rcu_read_lock() \
do { \
    preempt_disable(); \
    __acquire(RCU); \
    rcu_read_acquire(); \
} while(0)

include/linux/rcupdate.h
  
```

```

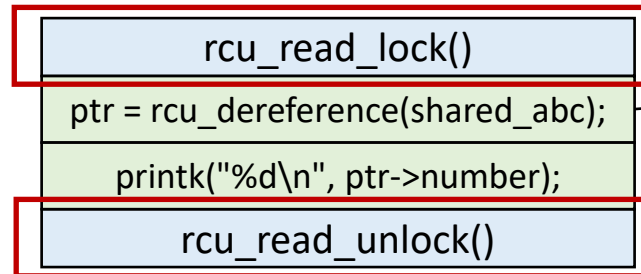
#define rcu_read_unlock() \
do { \
    rcu_read_release(); \
    __release(RCU); \
    preempt_enable(); \
} while(0)

include/linux/rcupdate.h
  
```

- `__acquire()` / `__release()`: sparse checker (semantic parser)
 - ✓ Sparse checking of RCU-protected pointers: Add `__rcu` marker
 - Reference: [PROPER CARE AND FEEDING OF RETURN VALUES FROM rcu_dereference\(\)](#)

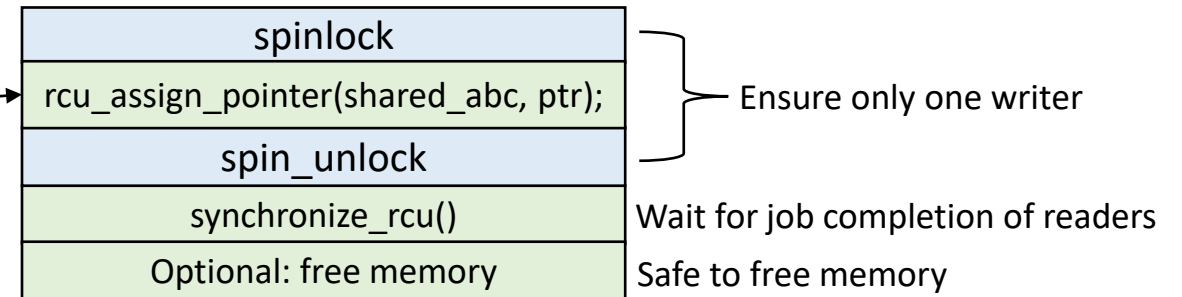
Reader: rcu_read_lock() & rcu_read_unlock()

RCU Reader



Is new data or old data read?

RCU Writer w/ valid pointer assignment



```
#define rcu_read_lock() \
do { \
    preempt_disable(); \
    __acquire(RCU); \
    rcu_read_acquire(); \
} while(0)

include/linux/rcupdate.h
```

```
#define rcu_read_unlock() \
do { \
    rcu_read_release(); \
    __release(RCU); \
    preempt_enable(); \
} while(0)

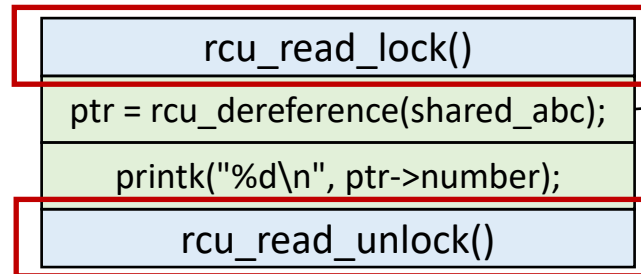
include/linux/rcupdate.h
```

- `rcu_read_acquire()` / `rcu_read_release()`: lock dep (runtime locking correctness validator)
 - ✓ Programming error (deadlock)

```
rcu_read_lock();
synchronize_rcu();
rcu_read_unlock();
```

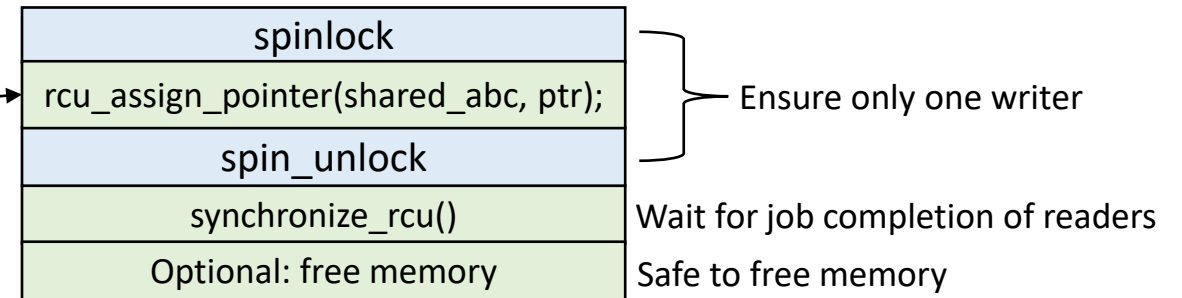
Reader: rcu_read_lock() & rcu_read_unlock()

RCU Reader



Is new data or old data read?

RCU Writer w/ valid pointer assignment



```

#define rcu_read_lock() \
    do { \
        preempt_disable(); \
        __acquire(RCU); \
        rcu_read_acquire(); \
    } while(0)
include/linux/rcupdate.h
  
```

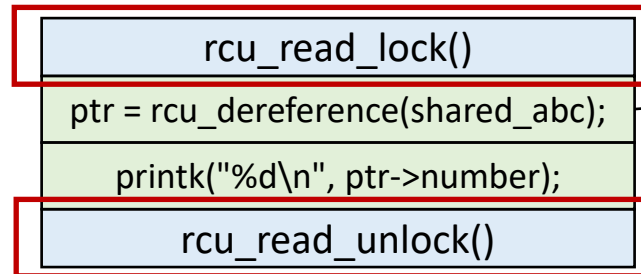
```

#define rcu_read_unlock() \
    do { \
        rcu_read_release(); \
        __release(RCU); \
        preempt_enable(); \
    } while(0)
include/linux/rcupdate.h
  
```

It's illegal to block while in an RCU read-side CS (Exception: CONFIG_PREEMPT_RCU=y)

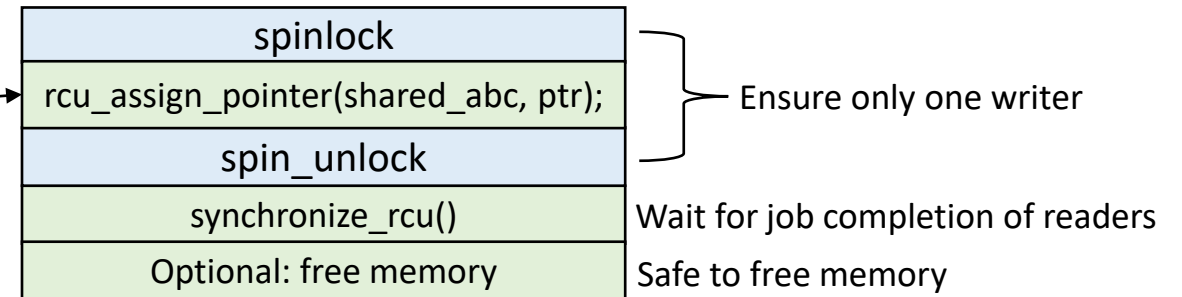
Reader: rcu_read_lock() & rcu_read_unlock()

RCU Reader



Is new data or old data read?

RCU Writer w/ valid pointer assignment



```

#define rcu_read_lock() \
do { \
    preempt_disable(); \
    __acquire(RCU); \
    rcu_read_acquire(); \
} while(0)
include/linux/rcupdate.h
  
```

```

#define rcu_read_unlock() \
do { \
    rcu_read_release(); \
    __release(RCU); \
    preempt_enable(); \
} while(0)
include/linux/rcupdate.h
  
```

rcu_assign_pointer() and rcu_dereference() invocations communicate spatial synchronization via stores to and loads from the RCU-protected pointer

Reader: rcu_dereference()

RCU Reader

```
rcu_read_lock()
```

```
ptr = rcu_dereference(shared_abc);
```

```
printk("%d\n", ptr->number);
```

```
rcu_read_unlock()
```

```
#ifdef CONFIG_SMP
#define smp_mb()      mb()
#define smp_rmb()     barrier()
#define smp_wmb()     barrier()
#define smp_read_barrier_depends()  do {} while(0)
#else
#define smp_mb()      barrier()
#define smp_rmb()     barrier()
#define smp_wmb()     barrier()
#define smp_read_barrier_depends()  do {} while(0)
#endif
include/asm-x86/system_64.h
```

```
/*
 * Prevent the compiler from merging or refetching accesses. The compiler
 * is also forbidden from reordering successive instances of ACCESS_ONCE(),
 * but only when the compiler is aware of some particular ordering. One way
 * to make the compiler aware of ordering is to put the two invocations of
 * ACCESS_ONCE() in different C statements.
 *
 * This macro does absolutely -nothing- to prevent the CPU from reordering,
 * merging, or refetching absolutely anything at any time.
 */
#define ACCESS_ONCE(x) (*(volatile typeof(x) *)&(x))

/**
 * rcu_dereference - fetch an RCU-protected pointer in an
 * RCU read-side critical section. This pointer may later
 * be safely dereferenced.
 *
 * Inserts memory barriers on architectures that require them
 * (currently only the Alpha), and, more importantly, documents
 * exactly which pointers are protected by RCU.
 */

#define rcu_dereference(p) ({ \
    typeof(p) __p1 = ACCESS_ONCE(p); \
    smp_read_barrier_depends(); \
    (__p1); \
})
```

```
include/linux/rcupdate.h
```

```
218,22-36
```

```
80%
```

Total Store Order (TSO) – x86 Memory Model: x86 is relatively strongly ordered system

- Preserve order
 - **Load -> Load**, Load -> Store and Store -> Store
- Might be re-order because of store buffer
 - Store -> Load

Writer: rcu_assign_pointer()

RCU Writer

spinlock
rcu_assign_pointer(shared_abc, ptr);
spin_unlock
synchronize_rcu()
Optional: free memory

```
/**
 * rcu_assign_pointer - assign (publicize) a pointer to a newly
 * initialized structure that will be dereferenced by RCU read-side
 * critical sections. Returns the value assigned.
 *
 * Inserts memory barriers on architectures that require them
 * (pretty much all of them other than x86), and also prevents
 * the compiler from reordering the code that initializes the
 * structure after the pointer assignment. More importantly, this
 * call documents which pointers will be dereferenced by RCU read-side
 * code.
 */

#define rcu_assign_pointer(p, v)      ({ \
                                     smp_wmb(); \
                                     (p) = (v); \
                                     })

include/linux/rcupdate.h 255,0-1
```

```
/* Optimization barrier */
/* The "volatile" is due to gcc bugs */
#define barrier() __asm__ __volatile__("" : : : "memory")

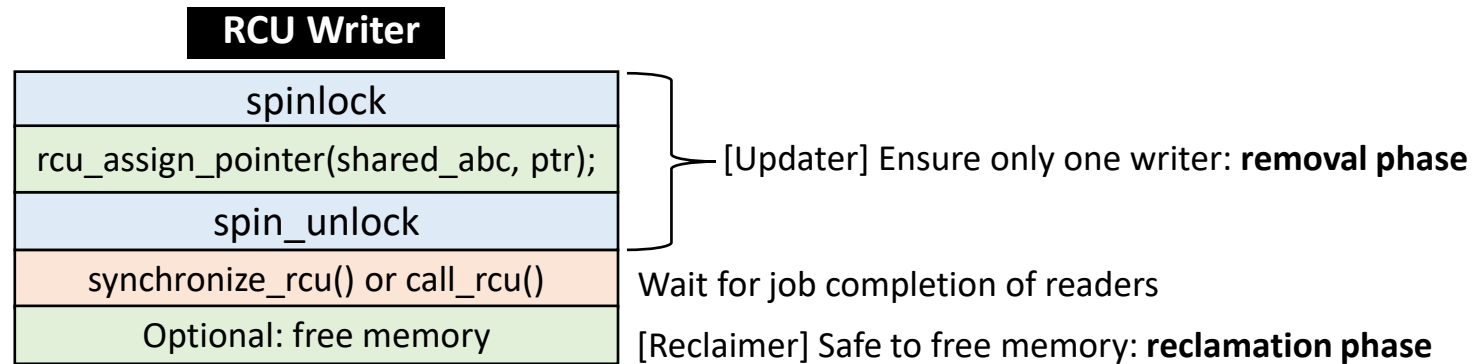
include/linux/compiler-gcc.h
```

```
#ifdef CONFIG_SMP
#define smp_mb()      mb()
#define smp_rmb()     barrier()
#define smp_wmb()     barrier()
#define smp_read_barrier_depends()    do {} while(0)
#else
#define smp_mb()      barrier()
#define smp_rmb()     barrier()
#define smp_wmb()     barrier()
#define smp_read_barrier_depends()    do {} while(0)
#endif
include/asm-x86/system_64.h
```

Total Store Order (TSO) – x86 Memory Model: x86 is relatively strongly ordered system

- Preserve order
 - Load -> Load, Load -> Store and Store -> Store
- Might be re-order because of store buffer
 - Store -> Load

Writer: synchronize_rcu() & call_rcu()



- Mark the end of updater code and the beginning of reclaimer code
- [Synchronous] synchronize_rcu()
 - ✓ Block until all pre-existing RCU read-side critical sections on all CPUs have completed
 - ✓ leverages call_rcu()
- [Asynchronous] call_rcu()
 - ✓ Queue a callback for invocation after a grace period
 - ✓ [Scenario]
 - It's illegal to block RCU updater
 - Update-side performance is critically important

Why the name RCU (read-copy update)?

RCU Writer

```
static inline int audit_upd_rule(struct audit_rule *rule,
                                struct list_head *list,
                                __u32 newaction,
                                __u32 newfield_count)
{
    struct audit_entry *e;
    struct audit_entry *ne;

    list_for_each_entry(e, list, list) {
        if (!audit_compare_rule(rule, &e->rule)) {
            ne = kmalloc(sizeof(*entry), GFP_ATOMIC);
            if (ne == NULL)
                return -ENOMEM;
            audit_copy_rule(&ne->rule, &e->rule);
            ne->rule.action = newaction;
            ne->rule.field_count = newfield_count;
            list_replace_rcu(&e->list, &ne->list);
            call_rcu(&e->rcu, audit_free_rule);
            return 0;
        }
    }
    return -EFAULT;          /* No matching rule */
}
```

Create a copy

Why the name RCU (read-copy update)?

RCU Writer

```
static inline int audit_upd_rule(struct audit_rule *rule,
                                struct list_head *list,
                                __u32 newaction,
                                __u32 newfield_count)
{
    struct audit_entry *e;
    struct audit_entry *ne;

    list_for_each_entry(e, list, list) {
        if (!audit_compare_rule(rule, &e->rule)) {
            ne = kmalloc(sizeof(*entry), GFP_ATOMIC);
            if (ne == NULL)
                return -ENOMEM;
            audit_copy_rule(&ne->rule, &e->rule);
            ne->rule.action = newaction;
            ne->rule.field_count = newfield_count;
            list_replace_rcu(&e->list, &ne->list);
            call_rcu(&e->rcu, audit_free_rule);
            return 0;
        }
    }
    return -EFAULT;      /* No matching rule */
}
```

Update the copy

Why the name RCU (read-copy update)?

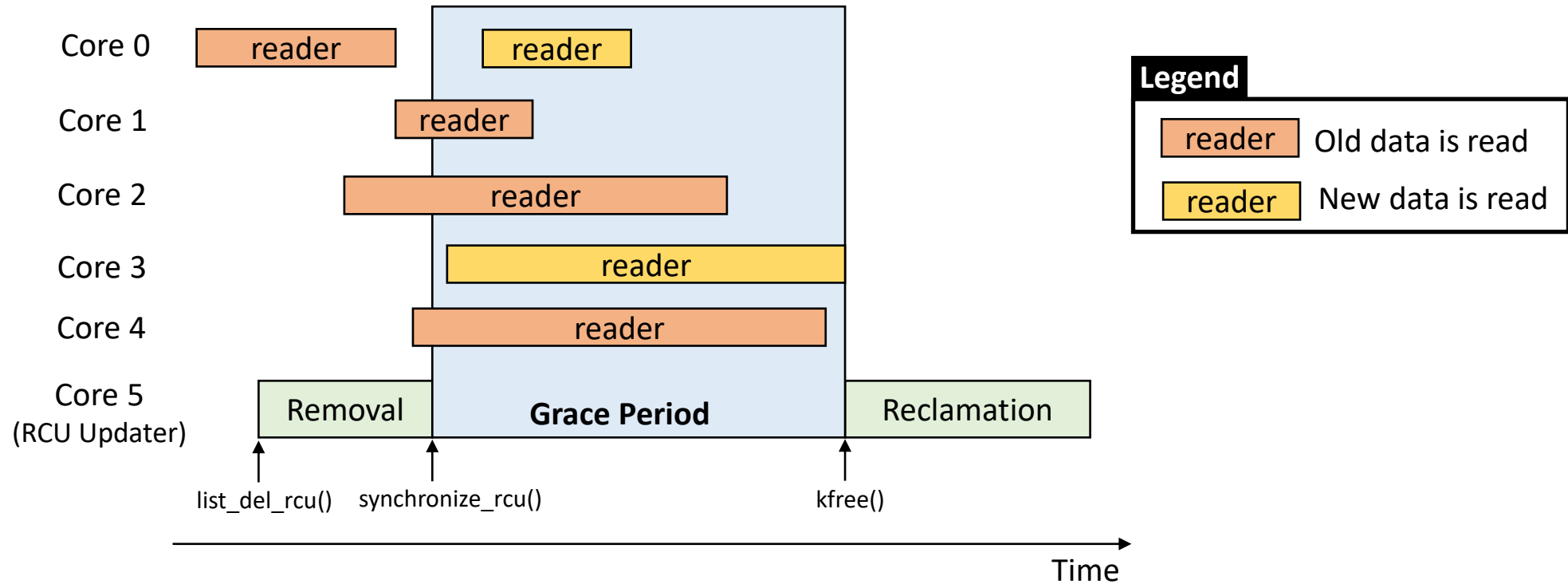
RCU Writer

```
static inline int audit_upd_rule(struct audit_rule *rule,
                                struct list_head *list,
                                __u32 newaction,
                                __u32 newfield_count)
{
    struct audit_entry *e;
    struct audit_entry *ne;

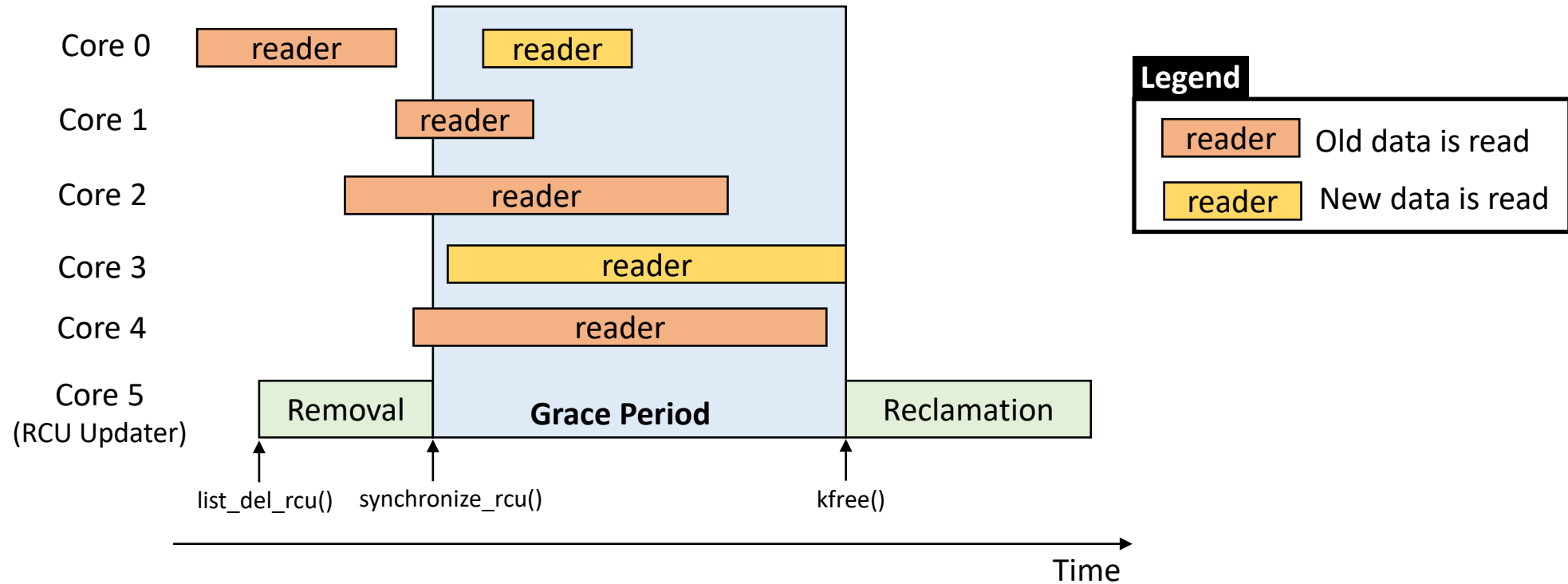
    list_for_each_entry(e, list, list) {
        if (!audit_compare_rule(rule, &e->rule)) {
            ne = kmalloc(sizeof(*entry), GFP_ATOMIC);
            if (ne == NULL)
                return -ENOMEM;
            audit_copy_rule(&ne->rule, &e->rule);
            ne->rule.action = newaction;
            ne->rule.field_count = newfield_count;
            list_replace_rcu(&e->list, &ne->list);
            call_rcu(&e->rcu, audit_free_rule);
            return 0;
        }
    }
    return -EFAULT;          /* No matching rule */
}
```

**Replace the old entry with
the newly created one**

RCU: Quiescent State & Grace Period



RCU: Quiescent State & Grace Period



Quotes from the book “Is Parallel Programming Hard, And, If So, What Can You Do About It?”

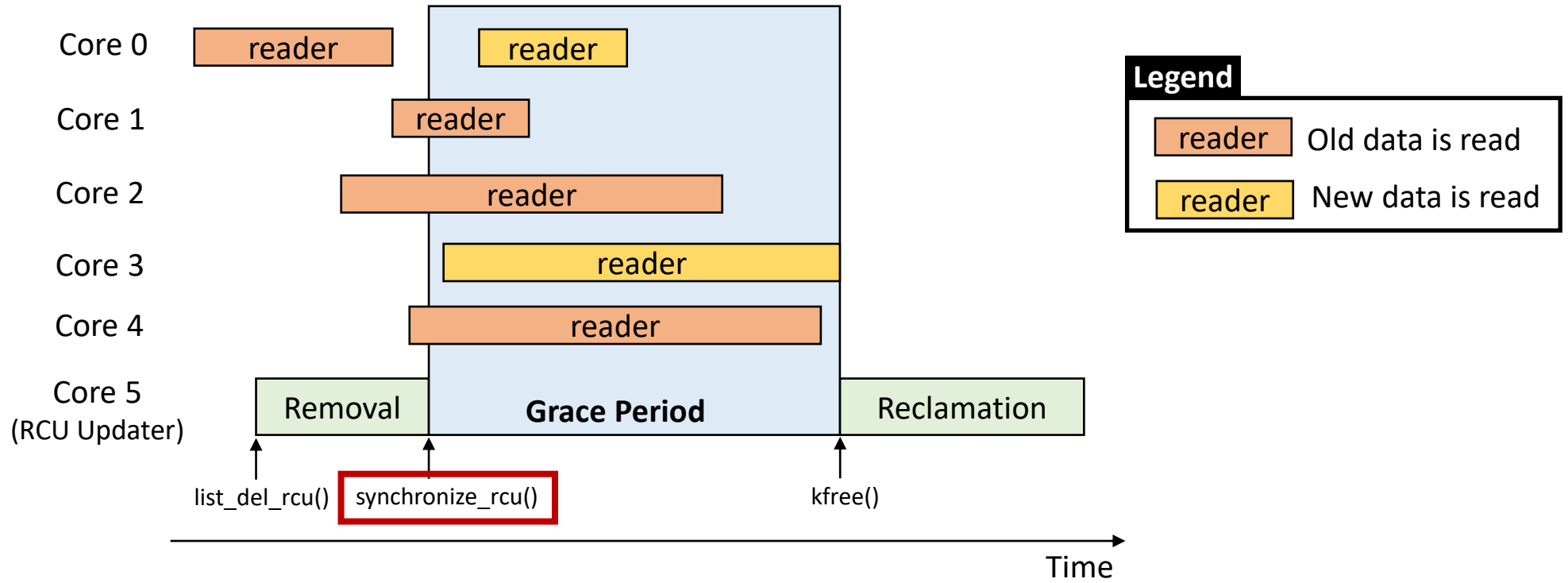
Quiescent State (QS)

- A point in the code where there can be no references held to RCU-protected data structures, which is normally any point outside of an RCU read-side critical section.
 - ✓ RCU read-side critical section is defined by the range between `rcu_read_lock()` and `rcu_read_unlock()`.

Grace Period (GP)

- All threads (cores) pass through at least one quiescent state.

RCU: Quiescent State & Grace Period



synchronize_rcu(): How to detect a grace period?

Grace Period: Waiting for readers - 6 Approaches

Approach #1:
Reference
Counter

Approach #2:
CPU Register

Approach #3:
Wait for a fixed
period time

Approach #4:
Wait forever

Approach #5:
Avoid the
period crashes

Approach #6: Quiescent-state-
based reclamation (QSBR)

Grace Period: Waiting for readers – Reference Counter

Approach #1:
Reference
Counter

Approach #2:
CPU Register

Approach #3:
Wait for a fixed
period time

Approach #4:
Wait forever

Approach #5:
Avoid the
period crashes

Approach #6: Quiescent-state-
based reclamation (QSBR)

Reference Counter

- Reference counter (share data) updated by `rcu_read_lock()` and `rcu_read_unlock()`
 - ✓ Scalability problem due to cache bouncing

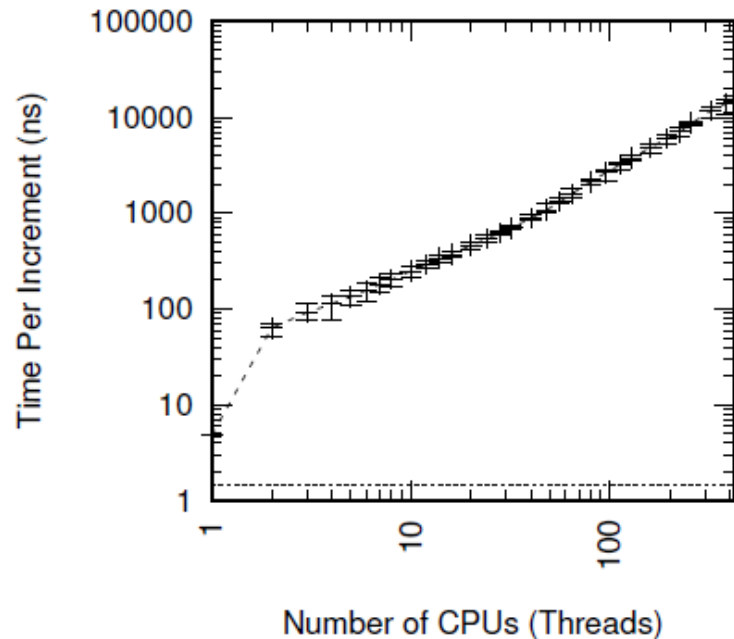


Figure 5.1: Atomic Increment Scalability on x86

* Figure reference from [Is Parallel Programming Hard, And, If So, What Can You Do About It?](#)

Grace Period: Waiting for readers – CPU Register



CPU Register

- Check CPUs' register (Prevent from accessing share data such as reference counter)
 - ✓ CPUs' register: Check each CPU's program counter (PC)
 - The updater polls each relevant PC. If the PC is not within read-side code, the corresponding CPU is within a quiescent state.
 - A complete grace period: All CPU's PCs have been observed to be outside of read-side code.
 - ✓ Challenges
 - Readers might invoke other functions
 - Code-motion optimization

Grace Period: Waiting for readers – Wait for a fixed period time



Wait for a fixed period time

- Wait the enough time to comfortably exceed the lifetime of any reasonable reader
 - ✓ Unreasonable reader → issue!

Grace Period: Waiting for readers – Wait forever

Approach #1:
Reference
Counter

Approach #2:
CPU Register

Approach #3:
Wait for a fixed
period time

Approach #4:
Wait forever

Approach #5:
Avoid the
period crashes

Approach #6: Quiescent-state-
based reclamation (QSBR)

Wait forever

- Accommodate the unreasonable reader.
- Bad reputation: leaking memory
 - ✓ Memory leaks often require untimely and inconvenient reboots.
 - ✓ Work well in a high-availability cluster where systems were periodically crashed in order to ensure that cluster really remained highly available.

Grace Period: Waiting for readers – Avoid the period crashes



Avoid the period crashes

- Covered by stop-the-world garbage collector
- In today's always-connected always-on world, stopping the world can gravely degrade response times.

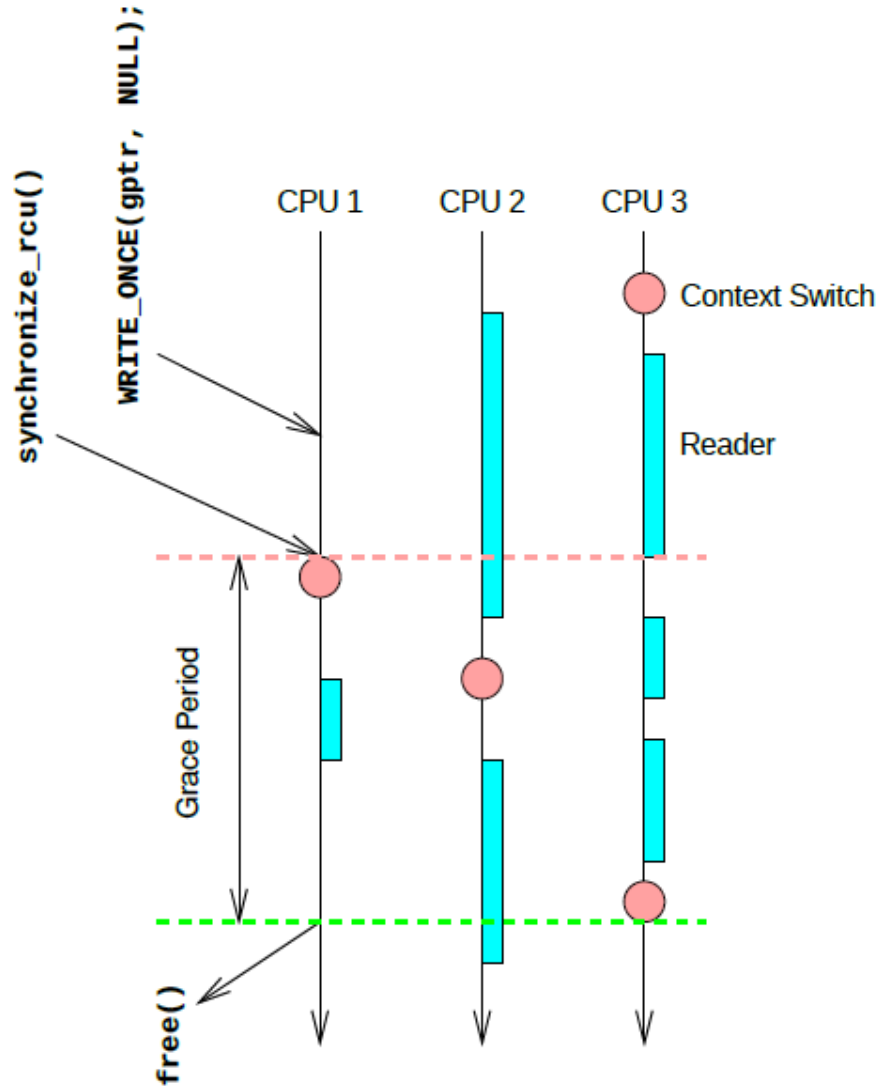
Grace Period: Waiting for readers – QSBR



QSBR

- Numerous applications already have states (termed **quiescent states**) that can be reached only after all pre-existing readers are done.
 - ✓ Transaction-processing application: the time between a pair of successive transactions might be a quiescent state.
 - ✓ Non-preemptive OS kernel: Context switch can be a quiescent state.
 - ✓ [Non-preemptive OS kernel] RCU reader must be prohibited from blocking while referencing a global data.

Grace Period: Waiting for readers – QSBR



[Concept] Implementation for non-preemptive Linux kernel

```
1 void synchronize_rcu(void)
2 {
3     int cpu;
4
5     for_each_online_cpu(cpu)
6         sched_setaffinity(current->pid, cpumask_of(cpu));
7 }
```

* [Not production quality] `sched_setaffinity()` function causes the current thread to execute on the specified CPU, which forces the destination CPU to execute a context switch.

Figure 9.8: QSBR: Waiting for Pre-Existing Readers

[Concept] QSBR: non-production and non-preemptible implementation

synchronize_rcu()

```
1 void synchronize_rcu(void)
2 {
3     int cpu;
4
5     for_each_online_cpu(cpu)
6         sched_setaffinity(current->pid, cpumask_of(cpu));
7 }
```

Force the destination CPU to execution switch: Completion of RCU reader

[Reference: Page #142 of Is Parallel Programming Hard, And, If So, What Can You Do About It?](#)

rcu_read_lock() & rcu_read_unlock() → disable/enable preemption

```
static inline void __rcu_read_lock(void)
{
    preempt_disable();
}

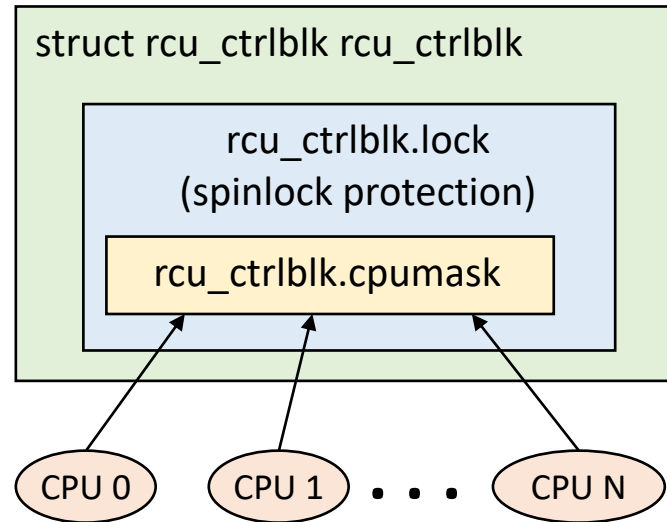
static inline void __rcu_read_unlock(void)
{
    preempt_enable();
    rcu_read_unlock_strict();
}

include/linux/rcupdate.h
```

Agenda

- [Overview] rwlock (reader-writer spinlock) vs RCU
- RCU Implementation Overview
 - ✓ High-level overview
 - ✓ RCU: List Manipulation – Old and New data
 - ✓ Reader/Writer synchronization: five basic APIs
 - Reader
 - rcu_read_lock() & rcu_read_unlock()
 - rcu_dereference()
 - Writer
 - rcu_assign_pointer()
 - synchronize_rcu() & call_rcu()
- **Classic RCU vs Tree RCU**
 - ✓ Will discuss implementation details about classic RCU ***only***
- RCU Flavors
- RCU Usage Summary & RCU Case Study

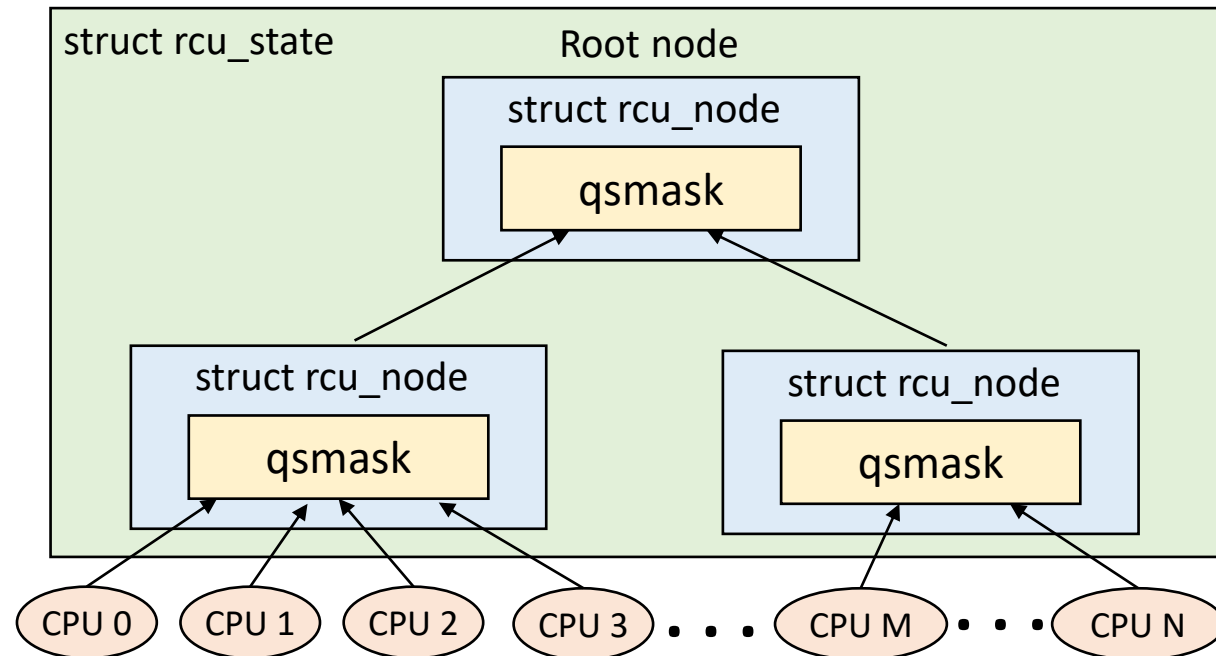
Classic RCU (< 2.6.29) and Tree RCU (>= 2.6.29)



Classic RCU

- Global cpumask: each bit indicates each core
- A grace period is complete if `rcu_ctrlblk.cpumask = 0`
- Scalability problem happens when the number of cores are increased
 - ✓ Lock contention: QS cores are contended to update `rcu_ctrlblk.cpumask`
 - ✓ Cache bouncing for frequent writes

Reference code: v2.6.24

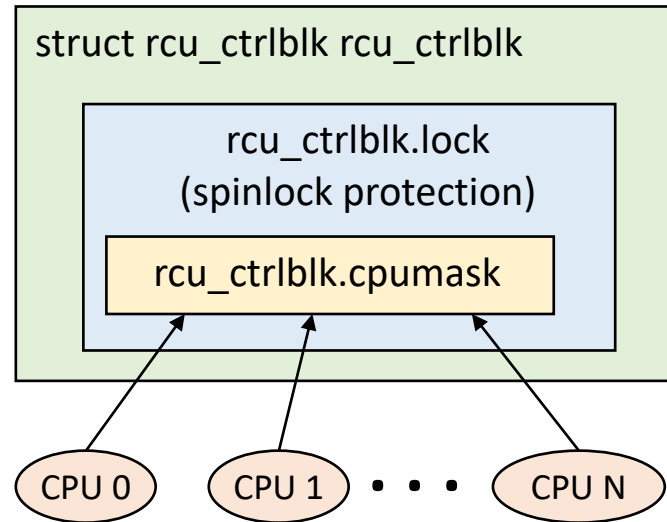


Tree RCU

- Group cpumask: reduce lock contention
- A grace period is complete if root's `rcu_node->qsmask = 0`
- Excellent scalability

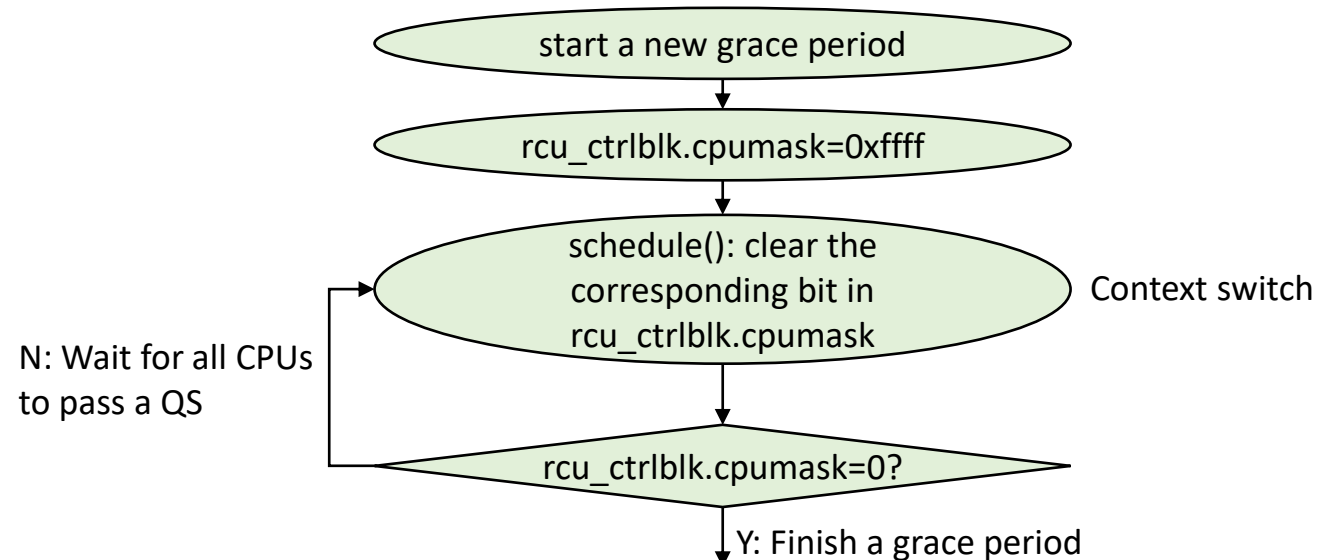
Reference code: v2.6.29

Classic RCU (< 2.6.29)

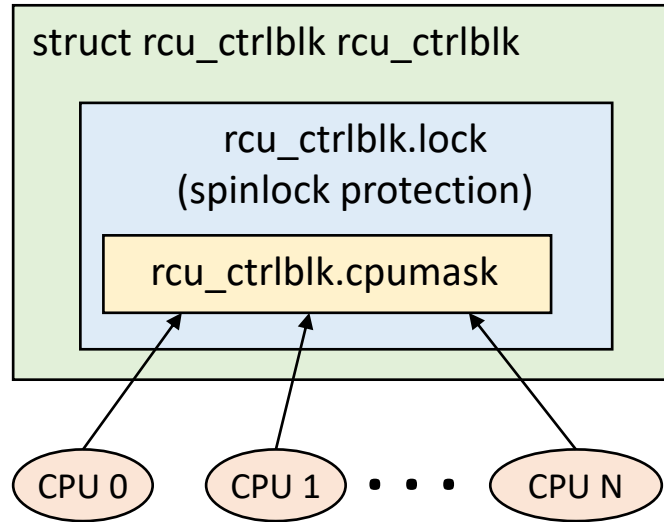


- Global cpumask: each bit indicates each core
- Scalability problem happens when the number of cores are increased
 - ✓ Lock contention
 - ✓ Cache bouncing for frequent writes

High-level concept: Suppose it's a 16-core system



Classic RCU: Data Structure



Global control block

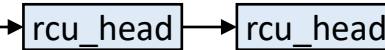
rcu_ctrlblk
cur
completed
next_pending
signaled
lock
cpumask

per-cpu

rcu_data	
qs handling	quiescbatch
	passed_quiesc
	qs_pending
batch handling	batch
	qlen
	# of queued callbacks
	cpu
	*nxtlist
	**nxttail
	*curlist
	**curtail
	*donelist
	**donetail

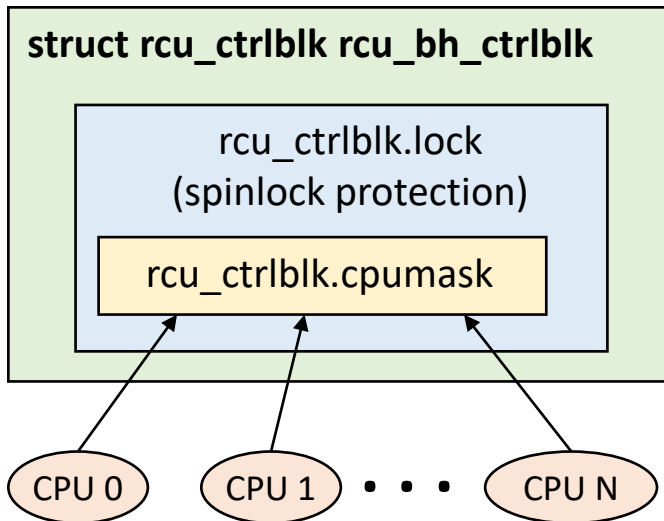
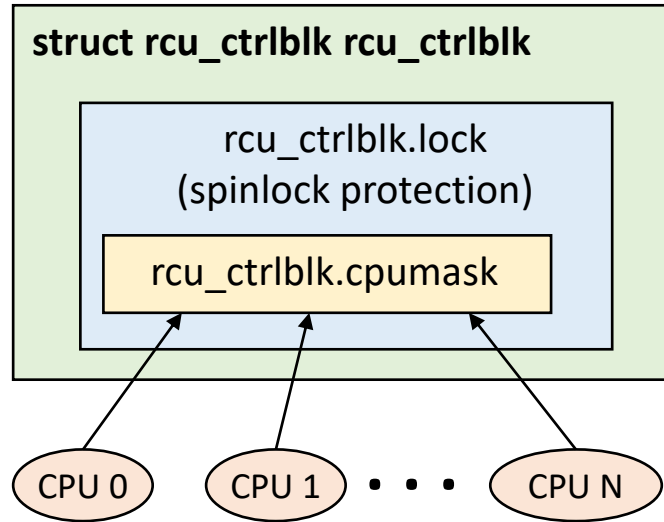
Callback supplied by call_rcu()

rcu_head
struct rcu_head *next
func



qs_pending: Avoid cacheline thrashing by assessing this percpu instead of cpumask bitmap

Classic RCU: start_kernel() -> rcu_init()



Global control block: rcu_ctrlblk

rcu_ctrlblk
cur = -300
completed = -300
next_pending
signaled
lock
cpumask = 0

Global control block: rcu_bh_ctrlblk

rcu_ctrlblk
cur = -300
completed = -300
next_pending
signaled
lock
cpumask = 0

① init percpu data

per-cpu: rcu_data

rcu_data	
qs handling	quiescbatch
	passed_quiesc
	qs_pending
batch handling	batch
	qlen
	# of queued callbacks
	cpu
	*nxtlist
	**nxttail
	*curlist
	**curtail
struct rcu_head	*donelist
	**donetail

② init percpu data

per-cpu: rcu_bh_data

rcu_data	
qs handling	quiescbatch
	passed_quiesc
	qs_pending
batch handling	batch
	qlen
	# of queued callbacks
	cpu
	*nxtlist
	**nxttail
	*curlist
	**curtail
struct rcu_head	*donelist
	**donetail

③ init rcu_tasklet

rcu_tasklet per-cpu

rcu_process_callbacks()

Classic RCU: start_kernel() -> rcu_init(): show “rcu_ctrlblk” only

[Global control block]
static struct rcu_ctrlblk rcu_ctrlblk;

rcu_ctrlblk
cur = -300
completed = -300
next_pending = 0
signaled = 0
lock
cpumask = 0

per-cpu variable initialized by rcu_init_percpu_data()

rcu_data	
qs handling	quiescbatch = rcp->completed = -300
	passed_quiesc = 0
	qs_pending = 0
batch handling	batch = 0
	qlen = 0 # of queued callbacks
	cpu = current cpu's ID
	blimit = 10 (default value)
	*nxtlist = NULL
	**nxttail
struct rcu_head	*curlist = NULL
	**curtail
	*donelist = NULL
	**donetail

address of

address of

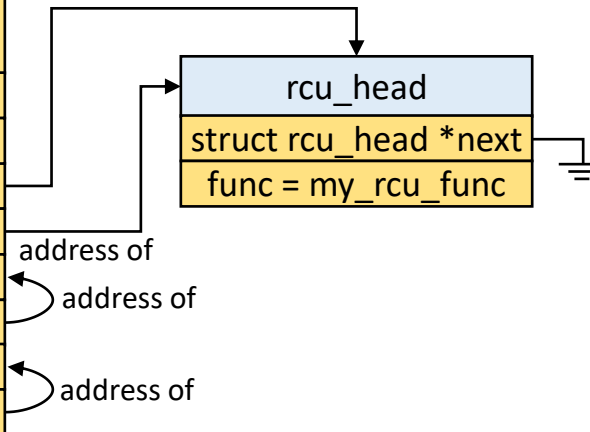
address of

Classic RCU: call_rcu()

per-cpu

rcu_data	
qs handling	quiescbatch = rcp->completed = -300
	passed_quiesc = 0
	qs_pending = 0
batch handling	batch = 0
	qlen = 0 1 # of queued callbacks
	cpu = current cpu's ID
	blimit = 10 (default value)
struct rcu_head	*nxtlist
	**nxttail
	*curlist = NULL
	**curtail
	*donelist = NULL
	**donetail

call_rcu(&my_rcu_head, my_rcu_func);



```
void fastcall call_rcu(struct rcu_head *head,
                      void (*func)(struct rcu_head *rcu))
{
    unsigned long flags;
    struct rcu_data *rdp;

    head->func = func;
    head->next = NULL;
    local_irq_save(flags);
    rdp = &__get_cpu_var(rcu_data);
    *rdp->nxttail = head;
    rdp->nxttail = &head->next;
    if (unlikely(++rdp->qlen > qhimark)) {
        rdp->blimit = INT_MAX;
        force_quiescent_state(rdp, &rcu_ctrlblk);
    }
    local_irq_restore(flags);
}
```

Classic RCU: synchronize_rcu() leverages call_rcu()

```
/* Because of FASTCALL declaration of complete, we use this wrapper */
static void wakeme_after_rcu(struct rcu_head *head)
{
    struct rcu_synchronize *rcu;

    rcu = container_of(head, struct rcu_synchronize, head);
    complete(&rcu->completion);
}

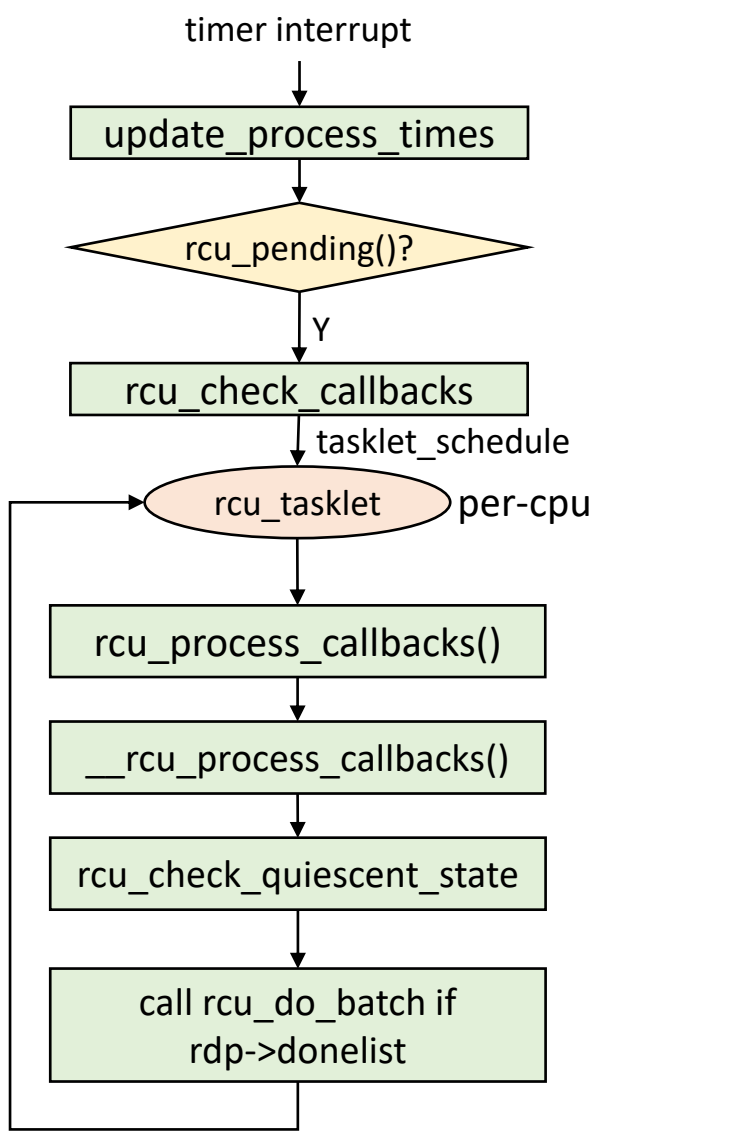
/**
 * synchronize_rcu - wait until a grace period has elapsed.
 *
 * Control will return to the caller some time after a full grace
 * period has elapsed, in other words after all currently executing RCU
 * read-side critical sections have completed. RCU read-side critical
 * sections are delimited by rcu_read_lock() and rcu_read_unlock(),
 * and may be nested.
 *
 * If your read-side code is not protected by rcu_read_lock(), do -not-
 * use synchronize_rcu().
 */
void synchronize_rcu(void)
{
    struct rcu_synchronize rcu;

    init_completion(&rcu.completion);
    /* Will wake me after RCU finished */
    call_rcu(&rcu.head, wakeme_after_rcu);

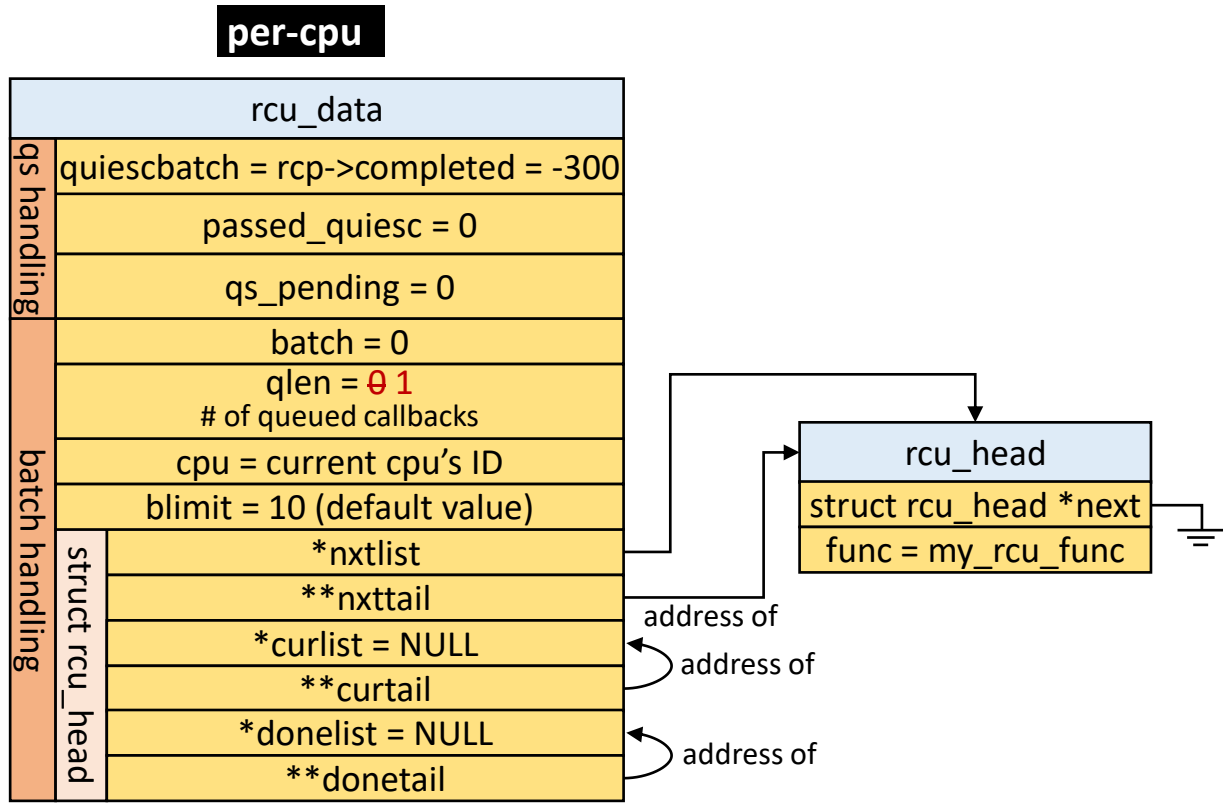
    /* Wait for it */
    wait_for_completion(&rcu.completion);
}

kernel/rcupdate.c
```

Classic RCU: Invoke rcu_pending() when a timer interrupt is triggered

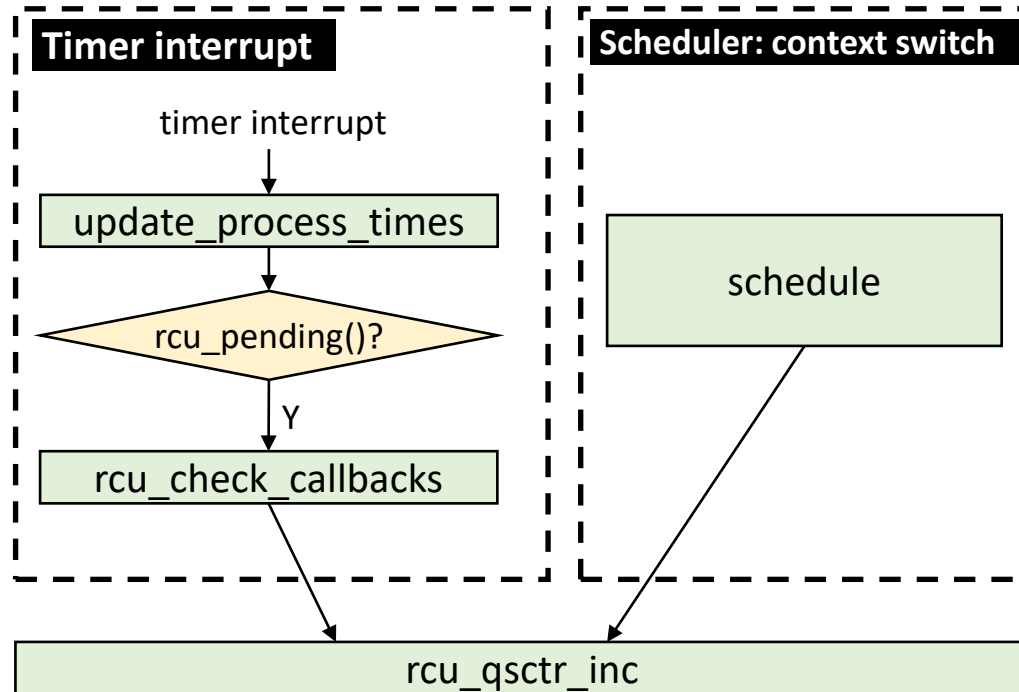


tasklet_schedule if rdp->donelist: All callbacks are not invoked yet due to rdp->blimit (limit on a processed batch)



```
static int __rcu_pending(struct rcu_ctrlblk *rcp, struct rcu_data *rdp)
{
+-- 6 lines: This cpu has pending rcu entries and the grace period-----
    /* This cpu has no pending entries, but there are new entries */
    if (!rdp->curlist && rdp->nxtlist)
        return 1;
+-- 9 lines: This cpu has finished callbacks to invoke -----
    /* nothing to do */
    return 0;
}
kernel/rcupdate.c 462,1-8
```

Classic RCU: Check if a CPU passes a QS



```
void rcu_check_callbacks(int cpu, int user)
{
    if (user ||
        (idle_cpu(cpu) && !in_softirq() &&
         hardirq_count() <= (1 << HARDIRQ_SHIFT))) {
        rcu_qsctr_inc(cpu);
        rcu_bh_qsctr_inc(cpu);
    } else if (!in_softirq())
        rcu_bh_qsctr_inc(cpu);
    tasklet_schedule(&per_cpu(rcu_tasklet, cpu));
}
```

kernel/rcupdate.c

514,19

```
/*
 * Increment the quiescent state counter.
 * The counter is a bit degenerated: We do not need to know
 * how many quiescent states passed, just if there was at least
 * one since the start of the grace period. Thus just a flag.
 */
```

```
static inline void rcu_qsctr_inc(int cpu)
{
    struct rcu_data *rdp = &per_cpu(rcu_data, cpu);
    rdp->passed_quiesc = 1;
}
```

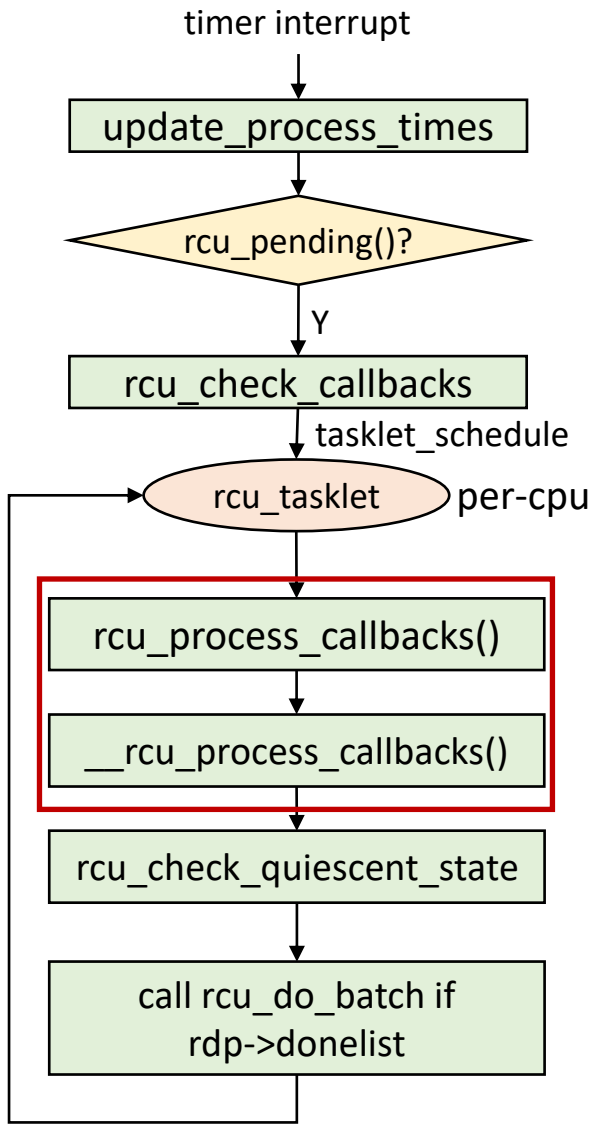
include/linux/rcupdate.h

103,7-14

```
asmlinkage void __sched schedule(void)
{
    +-- 9 lines: struct task_struct *prev, *next;
    rcu_qsctr_inc(cpu);
}
kernel/sched.c
```

[Note] “rcu_data.passed_quiesc = 1” does not clear the corresponding rcu_ctrlblk.cpumask directly. More checks are performed. See later slides.

Classic RCU: first timer interrupt



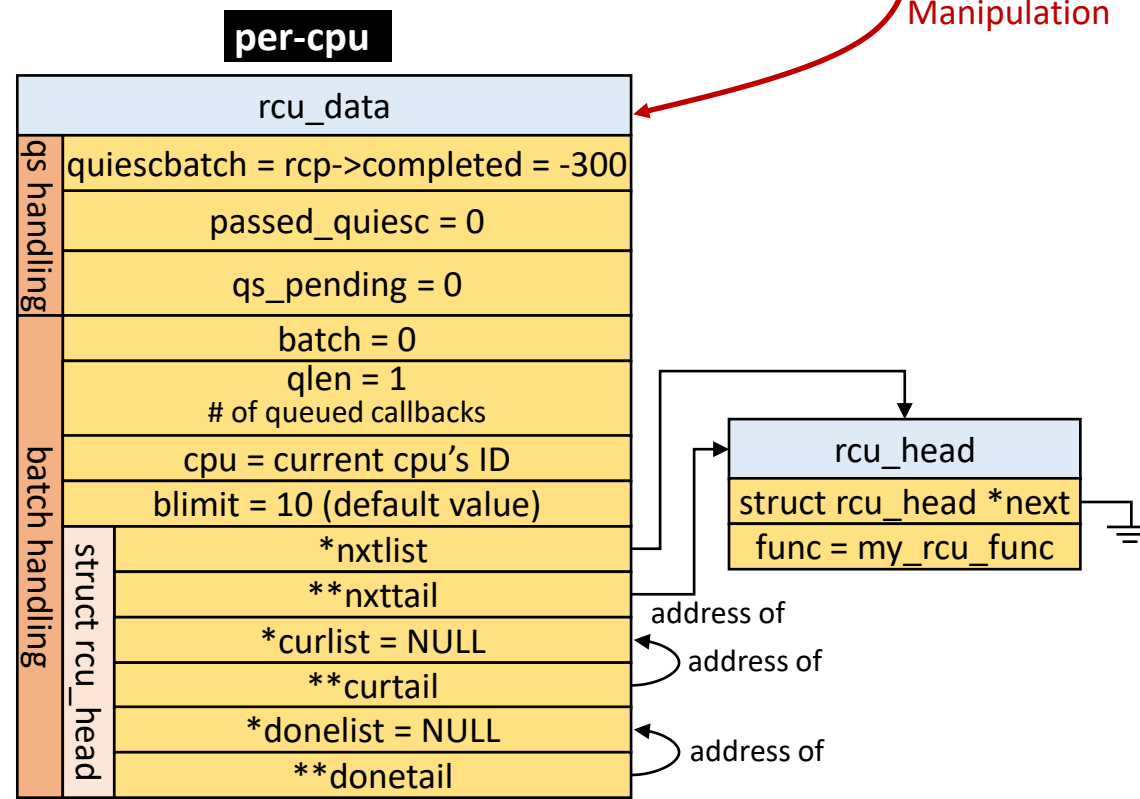
Kernel Source Reference: 2.6.24

[Global control block]
`static struct rcu_ctrlblk rcu_ctrlblk;`

rcu_ctrlblk
cur = -300
completed = -300
next_pending = 0
signaled = 0
lock
cpumask = 0

```
__rcu_process_callbacks()
{
    if (rdp->curlist &&
        !rcu_batch_before(rcp->completed, rdp->batch))
        move curlist to donetail

    if (rdp->nxtlist && !rdp->curlist) {
        move nxtlist to curlist
        rdp->batch = rcp->cur + 1;
        if (!rcp->next_pending) {
            rcp->next_pending = 1;
            rcu_start_batch(rcp);
        }
    }
    rcu_check_quiescent_state(rcp, rdp);
    if (rdp->donelist)
        rcu_do_batch(rdp);
}
```



Classic RCU

__rcu_process_callbacks()

static struct rcu_ctrlblk rcu_ctrlblk;

rcu_ctrlblk
cur = -300
completed = -300
next_pending = 0
signaled = 0
lock
cpumask = 0

per-cpu

rcu_data		
qs handling	quiescbatch = rcp->completed = -300	
	passed_quiesc = 0	
	qs_pending = 0	
batch handling	batch = 0	
	qlen = 1 # of queued callbacks	
	cpu = current cpu's ID	
	blimit = 10 (default value)	
	struct rcu_head	*nxtlist = NULL
		**nxttail
		*curlist = NULL
		**curtail
		*donelist = NULL
		**donetail

rcu_head
struct rcu_head *next
func = my_rcu_func

address of
address of

static struct rcu_ctrlblk rcu_ctrlblk;

rcu_ctrlblk
cur = -300
completed = -300
next_pending = 0 1
signaled = 0
lock
cpumask = 0

per-cpu

rcu_data		
qs handling	quiescbatch = rcp->completed = -300	
	passed_quiesc = 0	
	qs_pending = 0	
batch handling	batch = rcp->cur + 1 = -299	
	qlen = 1 # of queued callbacks	
	cpu = current cpu's ID	
	blimit = 10 (default value)	
	struct rcu_head	*nxtlist = NULL
		**nxttail
		*curlist
		**curtail
		*donelist = NULL
**donetail		

rcu_head
struct rcu_head *next
func = my_rcu_func

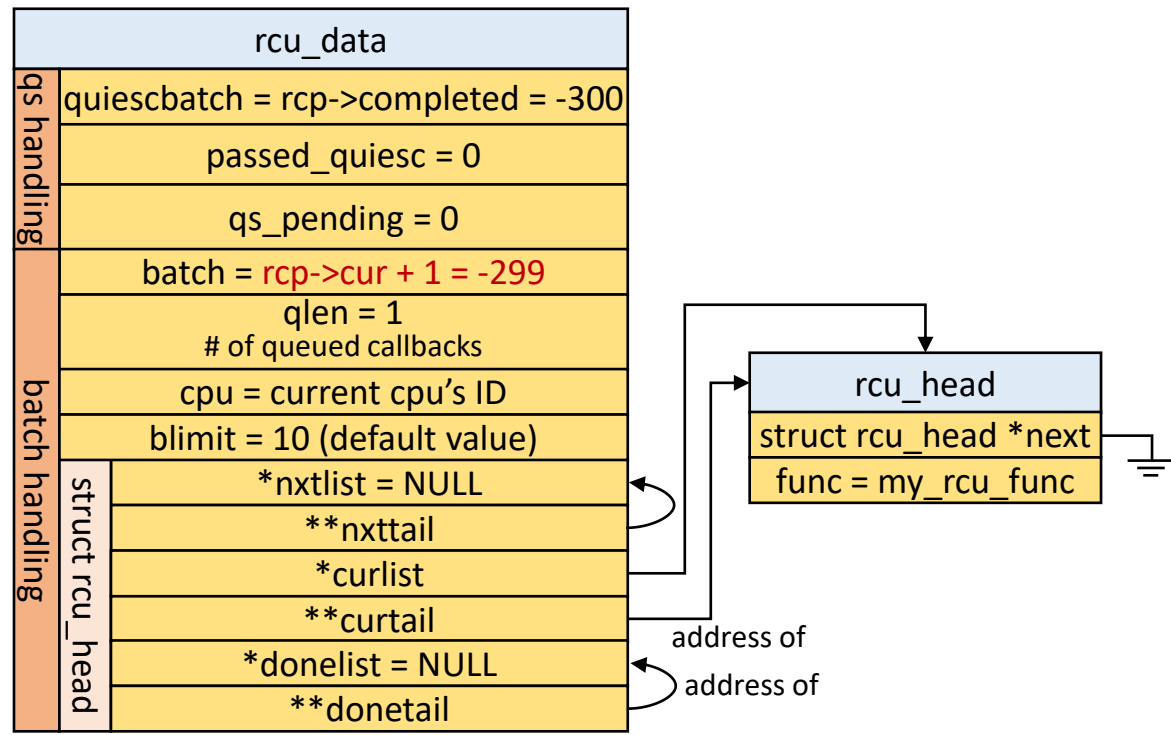
address of
address of

Classic RCU: first timer interrupt

static struct rcu_ctrlblk rcu_ctrlblk;

rcu_ctrlblk
cur = -300
completed = -300
next_pending = 0 1
signaled = 0
lock
cpumask = 0

per-cpu



```
_rcu_process_callbacks()
{
    if (rdp->curlist &&
        !rcu_batch_before(rcp->completed, rdp->batch))
        move curlist to donetail

    if (rdp->nxtlist && !rdp->curlist) {
        move nxtlist to curlist
        rdp->batch = rcp->cur + 1;
        if (!rcp->next_pending) {
            rcp->next_pending = 1;
            rcu_start_batch(rcp);
        }
    }
    rcu_check_quiescent_state(rcp, rdp);
    if (rdp->donelist)
        rcu_do_batch(rdp);
}
```

rcu_start_batch()

- A new grace period is started
- All cpus must go through a quiescent state
 - ✓ at least two calls to `rcu_check_quiescent_state()` are required
 - The first call: A new grace period is running
 - The second call: If there was a quiescent state, then
 1. Update `rcu_ctrlblk.cpumask`: Clear the corresponding CPU bit
 2. `rcu_ctrlblk.cpumask` is empty: the grace period is completed.

Classic RCU: first timer interrupt

```
static void rcu_start_batch(struct rcu_ctrlblk *rcp)
{
    if (rcp->next_pending &&
        rcp->completed == rcp->cur) {
        rcp->next_pending = 0;
+-- 4 lines: next_pending == 0 must be visible in-----
        smp_wmb();
        rcp->cur++;
+-- 6 lines: Accessing nohz_cpu_mask before incrementing rcp->cur needs
        smp_mb();
        cpus_andnot(rcp->cpumask, cpu_online_map, nohz_cpu_mask);

        rcp->signaled = 0;
    }
}
kernel/rcupdate.c 280,1
```

rcu_ctrlblk
cur = -300
completed = -300
next_pending = 1
signaled = 0
lock
cpumask = 0

rcu_ctrlblk
cur++ → -299
completed = -300
next_pending = 1 0
signaled = 0 0
lock
cpumask = cpu_online_map & ~nohz_cpu_mask

Classic RCU

rcu_start_batch()

static struct rcu_ctrlblk rcu_ctrlblk;

rcu_ctrlblk
cur = -300
completed = -300
next_pending = 1
signaled = 0
lock
cpumask = 0

per-cpu

rcu_data		
qs handling	quiescbatch = rcp->completed = -300	
	passed_quiesc = 0	
	qs_pending = 0	
batch handling	batch = -299	
	qlen = 1 # of queued callbacks	
	cpu = current cpu's ID	
	blimit = 10 (default value)	
	struct rcu_head	*nxtlist = NULL
		**nxttail
		*curlist
		**curtail
		*donelist = NULL
**donetail		

rcu_head
struct rcu_head *next
func = my_rcu_func

address of
address of

static struct rcu_ctrlblk rcu_ctrlblk;

rcu_ctrlblk
cur++ → -299
completed = -300
next_pending = 1 0
signaled = 0 0
lock
cpumask = 0 all_online_cpu_bitmask

per-cpu

rcu_data		
qs handling	quiescbatch = rcp->completed = -300	
	passed_quiesc = 0	
	qs_pending = 0	
batch handling	batch = -299	
	qlen = 1 # of queued callbacks	
	cpu = current cpu's ID	
	blimit = 10 (default value)	
	struct rcu_head	*nxtlist = NULL
		**nxttail
		*curlist
		**curtail
		*donelist = NULL
		**donetail

rcu_head
struct rcu_head *next
func = my_rcu_func

address of
address of

Classic RCU: first timer interrupt

```

_rcu_process_callbacks()
{
    if (rdp->curlist &&
        !rcu_batch_before(rcp->completed, rdp->batch))
        move curlist to donetail

    if (rdp->nxtlist && !rdp->curlist) {
        move nxtlist to curlist
        rdp->batch = rcp->cur + 1;
        if (!rcp->next_pending) {
            rcp->next_pending = 1;
            rcu_start_batch(rcp);
        }
    }
    rcu_check_quiescent_state(rcp, rdp);
    if (rdp->donelist)
        rcu_do_batch(rdp);
}

```

rcu_start_batch()

- A new grace period is started
- All cpus must go through a quiescent state
 - ✓ at least two calls to rcu_check_quiescent_state() are required
 - The first call: A new grace period is running
 - The second call: If there was a quiescent state, then
 1. Update rcu_ctrlblk.cpumask: Clear the corresponding CPU bit
 2. rcu_ctrlblk.cpumask is empty: the grace period is completed.

```

static void rcu_check_quiescent_state(struct rcu_ctrlblk *rcp,
                                     struct rcu_data *rdp)
{
    if (rdp->quiescbatch != rcp->cur) {
        /* start new grace period: */
        rdp->qs_pending = 1;
        rdp->passed_quiesc = 0;
        rdp->quiescbatch = rcp->cur;
        return;
    }

    /* Grace period already completed for this cpu?
     * qs_pending is checked instead of the actual bitmap to avoid
     * cacheline trashing.
     */
    if (!rdp->qs_pending)
        return;

    /*
     * Was there a quiescent state since the beginning of the grace
     * period? If no, then exit and wait for the next call.
     */
    if (!rdp->passed_quiesc)
        return;
    rdp->qs_pending = 0;

    spin_lock(&rcp->lock);
    /*
     * rdp->quiescbatch/rcp->cur and the cpu bitmap can come out of sync
     * during cpu startup. Ignore the quiescent state.
     */
    if (likely(rdp->quiescbatch == rcp->cur))
        cpu_quiet(rdp->cpu, rcp);

    spin_unlock(&rcp->lock);
}
kernel/rcupdate.c

```

The first call

The second call

Classic RCU (< 2.6.29): first timer interrupt

Kernel Source Reference: 2.6.24

```
static void rcu_check_quiescent_state(struct rcu_ctrlblk *rcp,
                                     struct rcu_data *rdp)
{
    if (rdp->quiescbatch != rcp->cur) {
        /* start new grace period: */
        rdp->qs_pending = 1;
        rdp->passed_quiesc = 0;
        rdp->quiescbatch = rcp->cur;
        return;
    }

    /* Grace period already completed for this cpu?
     * qs_pending is checked instead of the actual bitmap to avoid
     * cacheline trashing.
     */
    if (!rdp->qs_pending)
        return;

    /*
     * Was there a quiescent state since the beginning of the grace
     * period? If no, then exit and wait for the next call.
     */
    if (!rdp->passed_quiesc)
        return;
    rdp->qs_pending = 0;

    spin_lock(&rcp->lock);
    /*
     * rdp->quiescbatch/rcp->cur and the cpu bitmap can come out of sync
     * during cpu startup. Ignore the quiescent state.
     */
    if (likely(rdp->quiescbatch == rcp->cur))
        cpu_quiet(rdp->cpu, rcp);

    spin_unlock(&rcp->lock);
}
```

kernel/rcupdate.c 331,17

The first call, then wait for the next qs (context switch)

The second call

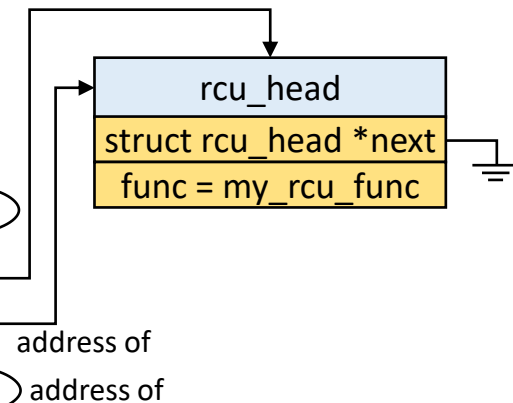
After the first call

static struct rcu_ctrlblk rcu_ctrlblk;

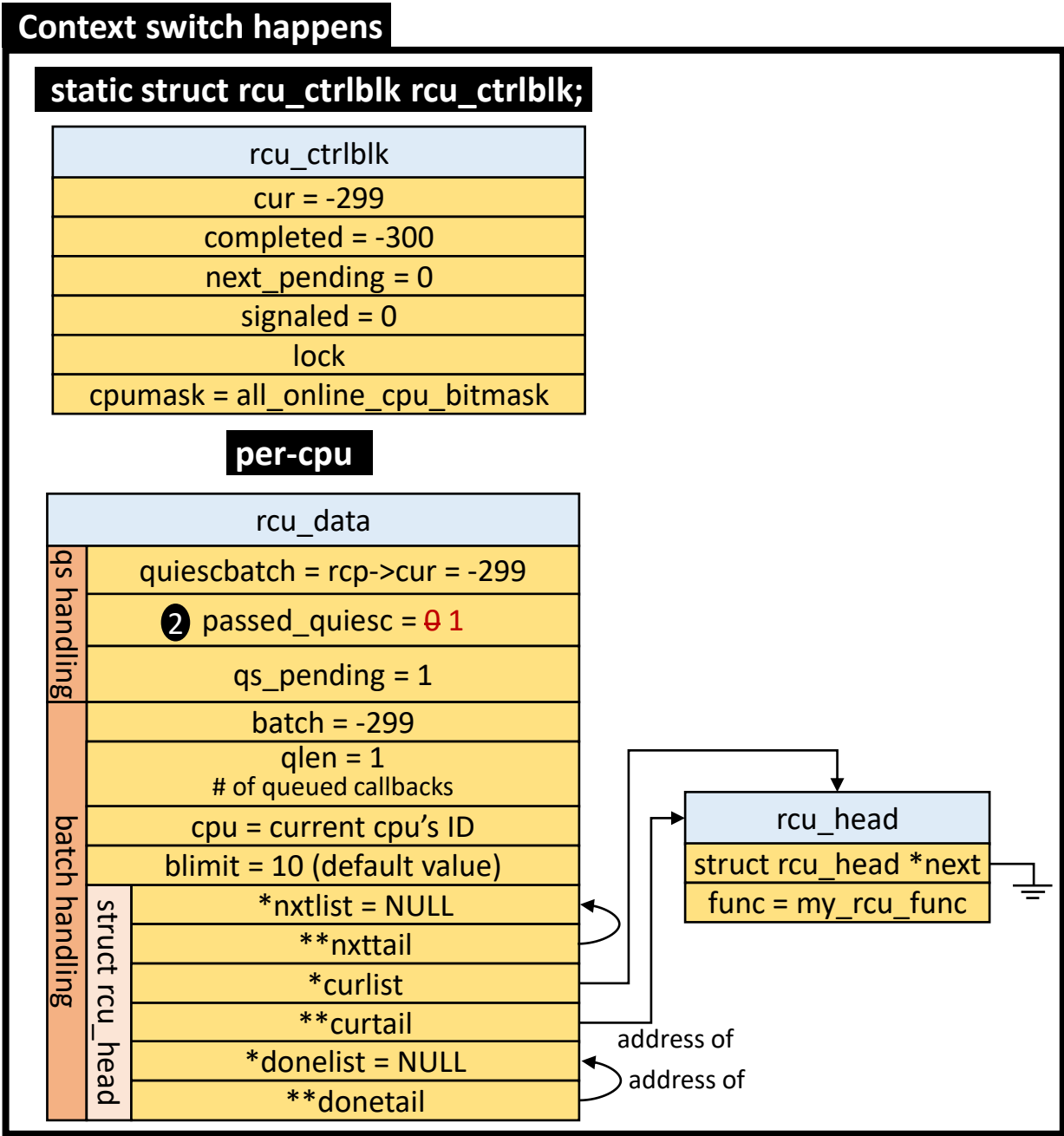
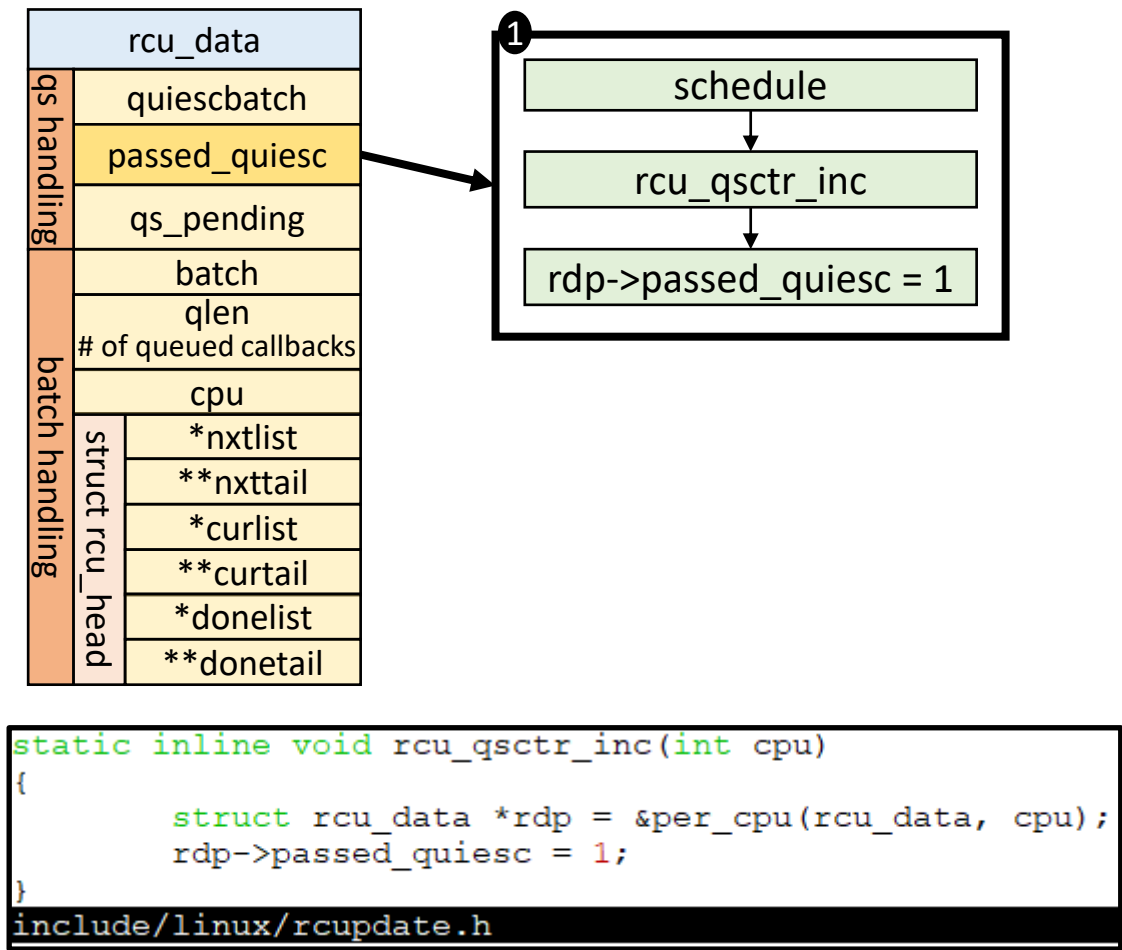
rcu_ctrlblk
cur = -299
completed = -300
next_pending = 0
signaled = 0
lock
cpumask = all_online_cpu_bitmask

per-cpu

rcu_data
qs handling
quiescbatch = rcp->cur = -299
passed_quiesc = 0
qs_pending = 1
batch handling
batch = -299
qlen = 1
of queued callbacks
cpu = current cpu's ID
blimit = 10 (default value)
struct rcu_head
*nxtlist = NULL
**nxttail
*curlist
**curtail
*donelist = NULL
**donetail



Classic RCU: first timer interrupt: quiescent state (context switch)



Classic RCU: second timer interrupt

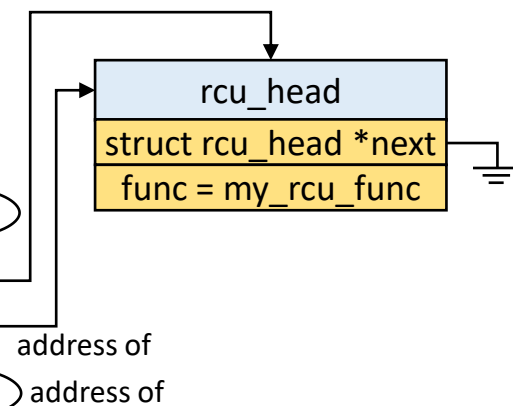
Kernel Source Reference: 2.6.24

static struct rcu_ctrlblk rcu_ctrlblk;

rcu_ctrlblk
cur = -299
completed = -300
next_pending = 0
signaled = 0
lock
cpumask = all_online_cpu_bitmask

per-cpu

rcu_data		
qs handling	quiescbatch = -299	
	passed_quiesc = 1	
	qs_pending = 1	
batch handling	batch = -299	
	qlen = 1 # of queued callbacks	
	cpu = current cpu's ID	
	blimit = 10 (default value)	
	struct rcu_head	*nxtlist = NULL
		**nxttail
		*curlist
		**curtail
		*donelist = NULL
		**donetail



Second timer interrupt -> __rcu_pending()

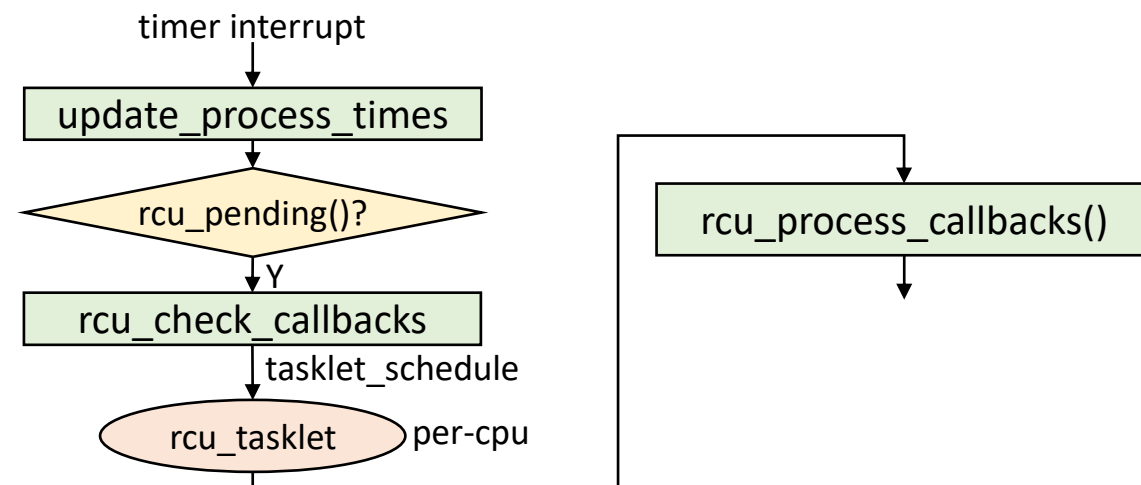
```
static int __rcu_pending(struct rcu_ctrlblk *rcp, struct rcu_data *rdp)
{
    /* This cpu has pending rcu entries and the grace period
     * for them has completed.
     */
    if (rdp->curlist && !rcu_batch_before(rcp->completed, rdp->batch))
        return 1;

    /* This cpu has no pending entries, but there are new entries */
    if (!rdp->curlist && rdp->nxtlist)
        return 1;

    /* This cpu has finished callbacks to invoke */
    if (rdp->donelist)
        return 1;

    /* The rcu core waits for a quiescent state from the cpu */
    if (rdp->quiescbatch != rcp->cur || rdp->qs_pending) True

    /* nothing to do */
    return 0;
}
kernel/rcupdate.c 474,21-28
```



Classic RCU

static struct rcu_ctrlblk rcu_ctrlblk;

rcu_ctrlblk
cur = -299
7 completed = cur = -299
next_pending = 0
signaled = 0
lock
cpumask = all_online_cpu_bitmask

5 **cpu_clear()**

per-cpu

rcu_data		
qs handling	quiescbatch = -299	
	passed_quiesc = 1	
	2 qs_pending = 1 0	
batch handling	batch = -299	
	qlen = 1 # of queued callbacks	
	cpu = current cpu's ID	
	blimit = 10 (default value)	
	struct rcu_head	*nxtlist = NULL
		**nxttail
		*curlist
		**curtail
		*donelist = NULL
		**donetail

rcu_head
struct rcu_head *next
func = my_rcu_func

address of

address of

```
static void rcu_check_quiescent_state(struct rcu_ctrlblk *rcp,
                                     struct rcu_data *rdp)
{
    if (rdp->quiescbatch != rcp->cur) {
        /* start new grace period: */
        rdp->qs_pending = 1;
        rdp->passed_quiesc = 0;
        rdp->quiescbatch = rcp->cur;
        return;
    }

    /* Grace period already completed for this cpu?
     * qs_pending is checked instead of the actual bitmap to avoid
     * cacheline trashing.
     */
    if (!rdp->qs_pending)
        return;

    /*
     * Was there a quiescent state since the beginning of the grace
     * period? If no, then exit and wait for the next call.
     */
    if (!rdp->passed_quiesc)
        return;

    1 rdp->qs_pending = 0;

    spin_lock(&rcp->lock);
    /*
     * rdp->quiescbatch/rcp->cur and the cpu bitmap can come out of sync
     * during cpu startup. Ignore the quiescent state.
     */
    3 if (likely(rdp->quiescbatch == rcp->cur))
        cpu_quiet(rdp->cpu, rcp);

    spin_unlock(&rcp->lock);
}
kernel/rcupdate.c 331,17
```

The second call

```
static void cpu_quiet(int cpu, struct rcu_ctrlblk *rcp)
{
    4 cpu_clear(cpu, rcp->cpumask);
    if (cpus_empty(rcp->cpumask)) {
        /* batch completed ! */
        6 All CPUs pass QS
        rcp->completed = rcp->cur;
        rcu_start_batch(rcp);
    }
}
kernel/rcupdate.c
```

Classic RCU: Second timer interrupt: rcu_start_batch()

```
static void cpu_quiet(int cpu, struct rcu_ctrlblk *rcp)
{
    cpu_clear(cpu, rcp->cpumask);
    if (cpus_empty(rcp->cpumask)) {
        /* batch completed ! */
        rcp->completed = rcp->cur;
        rcu_start_batch(rcp);
    }
}
kernel/rcupdate.c
```

```
static void rcu_start_batch(struct rcu_ctrlblk *rcp)
{
    if (rcp->next_pending &&
        rcp->completed == rcp->cur) {
        rcp->next_pending = 0;
+-- 4 lines: next_pending == 0 must be visible in-----
        smp_wmb();
        rcp->cur++;
+-- 6 lines: Accessing nohz_cpu_mask before incrementing rcp->cur needs
        smp_mb();
        cpus_andnot(rcp->cpumask, cpu_online_map, nohz_cpu_mask);

        rcp->signaled = 0;
    }
}
kernel/rcupdate.c 280,1
```

rcu_ctrlblk

cur = -299

completed = -299

next_pending = 0

signaled = 0

lock

cpumask = 0

rcu_ctrlblk

cur++ → -298

completed = -299

next_pending = 0

signaled = 0

lock

cpumask = 0 all_online_cpu_bitmask

Classic RCU: Third timer interrupt

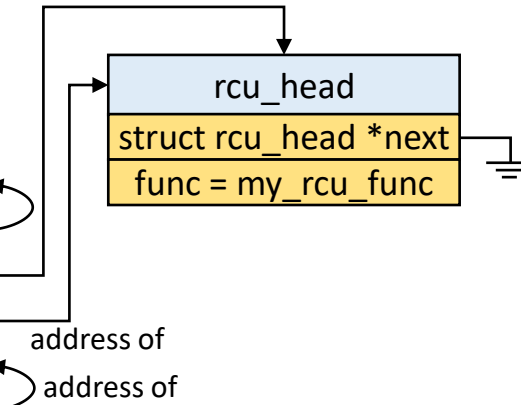
Kernel Source Reference: 2.6.24

static struct rcu_ctrlblk rcu_ctrlblk;

rcu_ctrlblk
cur++ = -298
completed = -299
next_pending = 0
signaled = 0
lock
cpumask = all_online_cpu_bitmask

rcu_data		
qs handling	quiescbatch = -299	
	passed_quiesc = 1	
	qs_pending = 0	
batch handling	batch = -299	
	qlen = 1 # of queued callbacks	
	cpu = current cpu's ID	
	blimit = 10 (default value)	
	struct rcu_head	*nxtlist = NULL
		**nxttail
		*curlist
		**curtail
		*donelist = NULL
		**donetail

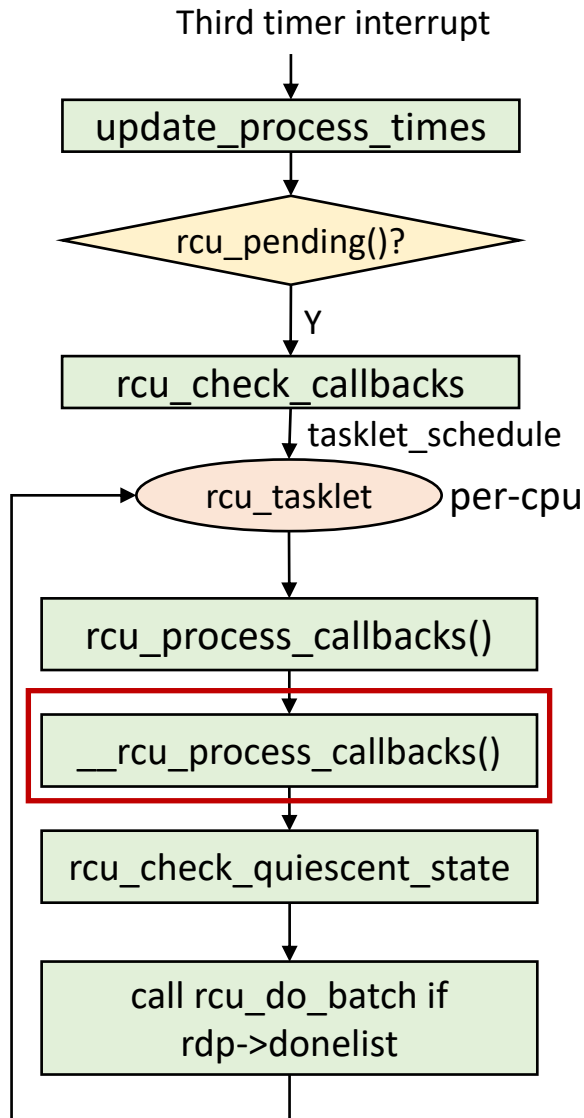
rcu_head
struct rcu_head *next
func = my_rcu_func



```
__rcu_process_callbacks()
{
    if (rdp->curlist &&
        !rcu_batch_before(rcp->completed, rdp->batch))
        move curlist to donetail

    if (rdp->nxtlist && !rdp->curlist) {
        move nxtlist to curlist
        rdp->batch = rcp->cur + 1;
        if (!rcp->next_pending) {
            rcp->next_pending = 1;
            rcu_start_batch(rcp);
        }
    }
    rcu_check_quiescent_state(rcp, rdp);
    if (rdp->donelist)
        rcu_do_batch(rdp);
}
```

```
/* Is batch a before batch b ? */
static inline int rcu_batch_before(long a, long b)
{
    return (a - b) < 0;
}
include/linux/rcupdate.h
```



Classic RCU: Third timer interrupt

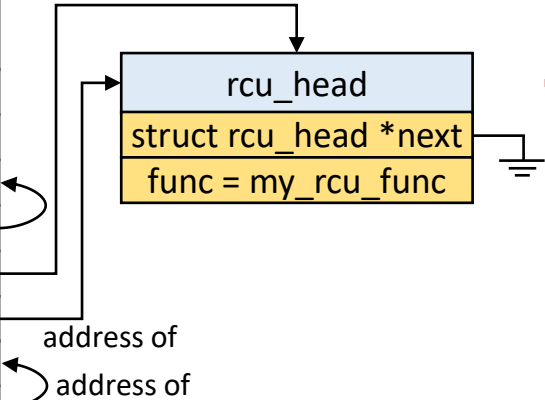
```
_rcu_process_callbacks()
{
    if (rdp->curlist &&
        !rcu_batch_before(rcp->completed, rdp->batch))
        move curlist to donetail

    if (rdp->nxtlist && !rdp->curlist) {
        move nxtlist to curlist
        rdp->batch = rcp->cur + 1;
        if (!rcp->next_pending) {
            rcp->next_pending = 1;
            rcu_start_batch(rcp);
        }
    }
    rcu_check_quiescent_state(rcp, rdp);
    if (rdp->donelist)
        rcu_do_batch(rdp);
}
```

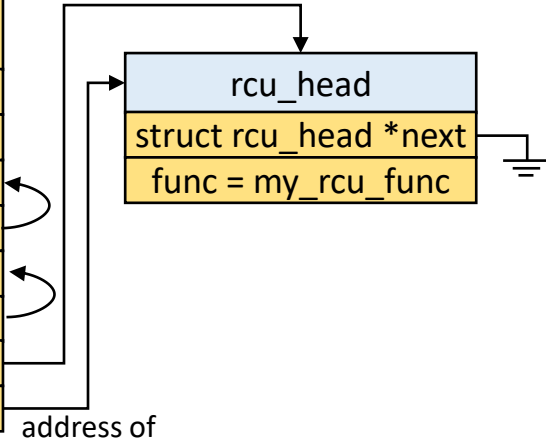
static struct rcu_ctrlblk rcu_ctrlblk;

rcu_ctrlblk
cur++ = -298
completed = -299
next_pending = 0
signaled = 0
lock
cpumask = all_online_cpu_bitmask

rcu_data		
qs handling	quiescbatch = -299	
	passed_quiesc = 1	
	qs_pending = 0	
batch handling	batch = -299	
	qlen = 1 # of queued callbacks	
	cpu = current cpu's ID	
	blimit = 10 (default value)	
	struct rcu_head	*nxtlist = NULL
		**nxttail
		*curlist
		**curtail
		*donelist = NULL
		**donetail



rcu_data		
qs handling	quiescbatch = -299	
	passed_quiesc = 1	
	qs_pending = 0	
batch handling	batch = -299	
	qlen = 1 # of queued callbacks	
	cpu = current cpu's ID	
	blimit = 10 (default value)	
	struct rcu_head	*nxtlist = NULL
		**nxttail
*curlist		
**curtail		
*donelist = NULL		
	**donetail	



Classic RCU: Third timer interrupt

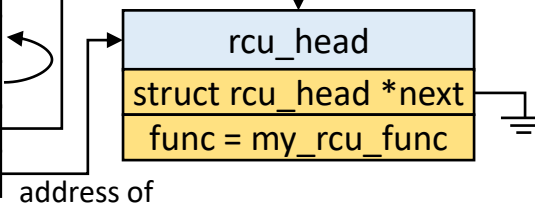
```
_rcu_process_callbacks()
{
    if (rdp->curlist &&
        !rcu_batch_before(rcp->completed, rdp->batch))
        move curlist to donetail

    if (rdp->nxtlist && !rdp->curlist) {
        move nxtlist to curlist
        rdp->batch = rcp->cur + 1;
        if (!rcp->next_pending) {
            rcp->next_pending = 1;
            rcu_start_batch(rcp);
        }
    }
    1 rcu_check_quiescent_state(rcp, rdp);
    if (rdp->donelist)
        rcu_do_batch(rdp);
}
```

rcu_data	
qs handling	5 quiescbatch = -299 = rcp->cur = -298
	4 passed_quiesc = 1 0
	3 qs_pending = 0 1
batch handling	batch = -299
	qlen = 1 # of queued callbacks
	cpu = current cpu's ID
	blimit = 10 (default value)
struct rcu_head	*nxtlist = NULL
	**nxttail
	*curlist = NULL
	**curtail
	*donelist
	**donetail

static struct rcu_ctrlblk rcu_ctrlblk;

rcu_ctrlblk
cur++ = -298
completed = -299
next_pending = 0
signaled = 0
lock
cpumask = all_online_cpu_bitmask



```
static void rcu_check_quiescent_state(struct rcu_ctrlblk *rcp,
                                     struct rcu_data *rdp)
{
    2 if (rdp->quiescbatch != rcp->cur) {
        /* start new grace period: */
        rdp->qs_pending = 1;
        rdp->passed_quiesc = 0;
        rdp->quiescbatch = rcp->cur;
        return;
    }

    /* Grace period already completed for this cpu?
     * qs_pending is checked instead of the actual bitmap to avoid
     * cacheline trashing.
     */
    if (!rdp->qs_pending)
        return;

    /*
     * Was there a quiescent state since the beginning of the grace
     * period? If no, then exit and wait for the next call.
     */
    if (!rdp->passed_quiesc)
        return;
    rdp->qs_pending = 0;

    spin_lock(&rcp->lock);
    /*
     * rdp->quiescbatch/rcp->cur and the cpu bitmap can come out of sync
     * during cpu startup. Ignore the quiescent state.
     */
    if (likely(rdp->quiescbatch == rcp->cur))
        cpu_quiet(rdp->cpu, rcp);

    spin_unlock(&rcp->lock);
}
kernel/rcupdate.c 331,17
```

Classic RCU: Third timer interrupt

Kernel Source Reference: 2.6.24

```
__rcu_process_callbacks()
{
    if (rdp->curlist &&
        !rcu_batch_before(rcp->completed, rdp->batch))
        move curlist to donetail

    if (rdp->nxtlist && !rdp->curlist) {
        move nxtlist to curlist
        rdp->batch = rcp->cur + 1;
        if (!rcp->next_pending) {
            rcp->next_pending = 1;
            rcu_start_batch(rcp);
        }
    }
    rcu_check_quiescent_state(rcp, rdp);
    if (rdp->donelist)
        rcu_do_batch(rdp);
}
```

1

```
static void rcu_do_batch(struct rcu_data *rdp)
{
    struct rcu_head *next, *list;
    int count = 0;

    list = rdp->donelist;
    while (list) {
        next = list->next;
        prefetch(next);
        list->func(list);
        list = next;
        if (++count >= rdp->blimit)
            break;
    }
    +-- 12 lines: rdp->donelist = list;-----
}

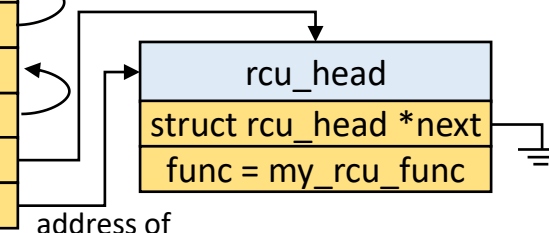
kernel/rcupdate.c
```

static struct rcu_ctrlblk rcu_ctrlblk;

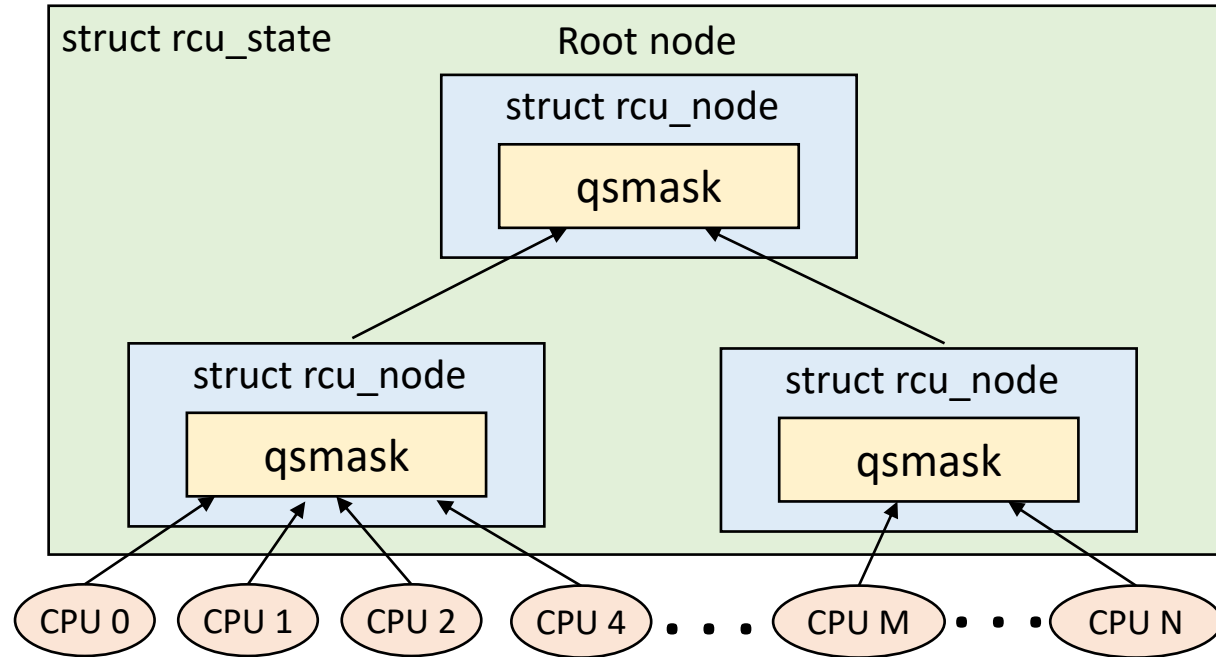
rcu_ctrlblk
cur++ = -298
completed = -299
next_pending = 0
signaled = 0
lock
cpumask = all_online_cpu_bitmask

rcu_data		
qs handling	quiescbatch = rcp->cur = -298	
	passed_quiesc = 1 0	
	qs_pending = 0 1	
batch handling	batch = -299	
	qlen = 1 # of queued callbacks	
	cpu = current cpu's ID	
	blimit = 10 (default value)	
	struct rcu_head	*nxtlist = NULL
		**nxttail
		*curlist = NULL
		**curtail
		2 *donelist
		**donetail

3 Invoke callbacks: depend on rdp->blimit (Check function 'rcu_do_batch' for detail)



Tree RCU (>= 2.6.29)



Tree RCU

- Group cpumask: reduce lock contention
- A grace period is complete if root's `rcu_node->qsmask = 0`
- Excellent scalability

Won't describe implementation detail about tree RCU. (kinda similar to classic RCU)

Agenda

- [Overview] rwlock (reader-writer spinlock) vs RCU
- RCU Implementation Overview
 - ✓ High-level overview
 - ✓ RCU: List Manipulation – Old and New data
 - ✓ Reader/Writer synchronization: five basic APIs
 - Reader
 - rcu_read_lock() & rcu_read_unlock()
 - rcu_dereference()
 - Writer
 - rcu_assign_pointer()
 - synchronize_rcu() & call_rcu()
- Classic RCU vs Tree RCU
- **RCU Flavors**
- RCU Usage Summary & RCU Case Study

RCU Flavors

- Vanilla RCU
- Bottom-half Flavor (Historical)
- Sched Flavor (Historical)
- Sleepable RCU (SRCU)
- Tasks RCU
- Tasks Rude RCU
- Tasks Trace RCU

RCU Flavors: Vanilla RCU (Classic RCU and Tree RCU)

- Reader RCU APIs

- ✓ `rcu_read_lock()` / `rcu_read_unlock()`
- ✓ `rcu_dereference()`

- Writer RCU APIs

- ✓ `rcu_assign_pointer()`
- ✓ Synchronous grace-period wait primitive: `synchronize_rcu()`
- ✓ Asynchronous grace-period wait primitive: `call_rcu()`

RCU Flavors: Bottom-half Flavor (Historical)

- Networking data structures that may be subjected to remote denial-of-service attacks.
 - ✓ CPUs never exit softirq execution due to DoS attacks.
 - Prevent CPUs from executing a context switch → prevent grace periods from ever ending.
 - Out-of-memory and a system hang
 - ✓ Disabling bottom half can prevent the issue.
- Reader RCU APIs
 - ✓ `rcu_read_lock_bh()` / `rcu_read_unlock_bh()`
 - `local_bh_disable()` / `local_bh_enable()`
 - ✓ `rcu_dereference_bh()`
- Writer: No change

RCU Flavors: Sched Flavor (Historical)

- Before preemptible RCU, context switch is a quiescent state
 - ✓ [A complete grace period] Need to wait for all pre-existing interrupt and NMI handler
- [CONFIG_PREEMPTION=n]
 - ✓ Vanilla RCU and RCU-sched grace period waits for pre-existing interrupt and NMI handlers
 - ✓ Vanilla RCU and RCU-sched have identical implementations
- [CONFIG_PREEMPTION=Y] for RCU-sched
 - ✓ Preemptible RCU does not need to wait for pre-existing interrupt and NMI handler.
 - ✓ The code outside of an RCU read-side critical section → a QS
 - ✓ rcu_read_lock_sched() → disable preemption
 - ✓ rcu_read_unlock_sched() → re-enable preemption
 - ✓ A preemption attempt during the RCU-sched read-side critical section:
 - ✓ rcu_read_unlock_sched() will enter the scheduler
- Reader RCU APIs
 - ✓ rcu_read_lock_sched() / rcu_read_unlock_sched()
 - ✓ preempt_disable() / preempt_enable()
 - ✓ local_irq_save() / local_irq_restore()
 - ✓ hardirq enter / hardirq exit
 - ✓ NMI enter / NMI exit
 - ✓ rcu_dereference_sched()
- Writer: No change

RCU Flavors: Sleepable RCU (SRCU)

- Classic RCU: blocking or sleeping is strictly prohibited
- SRCU: Allow arbitrary sleeping (or blocking) within RCU read-side critical section
 - ✓ Real-time kernel
 - Require that spinlock critical section be preemptible
 - Require that RCU read-side critical section be preemptible
 - ✓ Extend grace periods
- Different domains (srcu_struct structure) are defined
 - Benefit: a slow SRCU reader in one domain does not delay a SRCU grace period in another domain

```
struct srcu_struct ss;  
int idx;  
  
idx = srcu_read_lock(&ss);  
do_something();  
srcu_read_unlock(&ss, idx);
```

RCU Flavors: Tasks RCU

- RCU mechanism

- ✓ Keep old version of data structure until **no CPU** holds a reference to it. → the structure can be freed (context switch happens).

- Tasks RCU

- ✓ Defer the destruction of an old data structure until it is known that **no process** holds a reference to it
- ✓ Scenario: Handle the trampolines used in Linux-kernel tracing
 - Tracer subsystem: ftrace and kprobe
 - Kprobe: [Return probe – Trampoline](#)
- ✓ Reader RCU APIs
 - No explicit read-side marker
 - Voluntary context switches separate successive Tasks RCU read-side critical sections.
- ✓ Writer RCU APIs
 - Synchronous grace-period-wait primitives: `synchronize_rcu_tasks()`
 - Asynchronous grace-period-wait primitives: `call_rcu_tasks()`

RCU Flavors: Tasks Rude RCU

- Tasks RCU does not wait for idle tasks
 - ✓ Idle tasks: do not run voluntary context switches
 - Remain idle for long periods of time
 - ✓ Cannot work for tracing of code within idle loop
- Tasks Rude RCU
 - ✓ Scenario: Trampoline that might be involved in tracing of code within the idle loop
 - ✓ Reader RCU APIs
 - No explicit read-side marker
 - Preemption-disabled region of code is a Tasks Rude RCU reader
 - ✓ Writer RCU APIs
 - Synchronous grace-period-wait primitives: `synchronize_rcu_tasks_rude()`
 - Asynchronous grace-period-wait primitives: `call_rcu_tasks_rude()`

RCU Flavors: Tasks Trace RCU

- Tasks RCU and Tasks Rude RCU disallow sleeping while executing in a given trampoline
- Tasks Rude RCU
 - ✓ Scenario: BPF programs need to sleep
 - ✓ Reader RCU APIs
 - Explicit read-side marker: `rcu_read_lock_trace()` / `rcu_read_unlock_trace()`
 - ✓ Writer RCU APIs
 - Synchronous grace-period-wait primitives: `synchronize_rcu_tasks_trace()`
 - Asynchronous grace-period-wait primitives: `call_rcu_tasks_trace()`

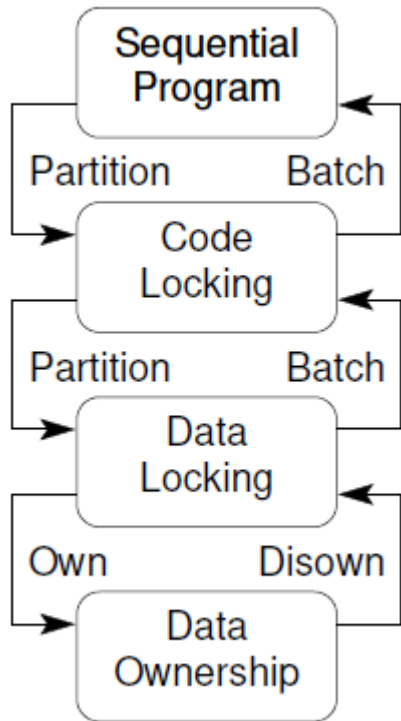
Agenda

- [Overview] rwlock (reader-writer spinlock) vs RCU
- RCU Implementation Overview
 - ✓ High-level overview
 - ✓ RCU: List Manipulation – Old and New data
 - ✓ Reader/Writer synchronization: five basic APIs
 - Reader
 - rcu_read_lock() & rcu_read_unlock()
 - rcu_dereference()
 - Writer
 - rcu_assign_pointer()
 - synchronize_rcu() & call_rcu()
- Classic RCU vs Tree RCU
- RCU Flavors
- RCU Usage Summary & RCU Case Study

RCU Usage Summary

Usage	Description	Applicable?
Routing Table	1. Read mostly 2. Stale and inconsistent data is permissible	Work great
Linux kernel's mapping from user-level System-V semaphore IDs to in-kernel data structures	1. Read mostly: Semaphore tends to be used far more frequently than they are created and destroyed. 2. Need consistent data: perform a semaphore operation on a semaphore that has already been deleted.	Work well
dentry cache in Linux kernel	1. Read/Write workload 2. Need consistent data	Might be ok
SLAB_TYPESAFE_BY_RCU slab-allocator flag provides type-safe memory to RCU readers	1. Write mostly 2. Need consistent data	Not best

Case Study: Design Patterns and Lock Granularity



- Code locking: use global locks only
 - ✓ Lock contention & scalability issue
- Data locking
 - ✓ Many data structures may be partitioned
 - Example: hash table
 - ✓ Each partition of data structure has its own lock
 - ✓ Improve lock contention & better scalability
- Data ownership
 - ✓ Data structure is partitioned over threads or CPUs
 - Each thread/CPU accesses its subset of data structure without any synchronization overhead
 - The most-heavily used data owned by a single CPU → Hot spot in this CPU
 - No sharing is required -> achieve ideal performance
 - Example: percpu variables in Linux kernel

Case Study: Code locking: dentry lookup

```
struct dentry * d_lookup(struct dentry * parent, struct qstr * name)
{
    struct dentry * dentry;
    spin_lock(&dcache_lock);
    dentry = __d_lookup(parent, name);
    if (dentry)
        __dget_locked(dentry);
    spin_unlock(&dcache_lock);
    return dentry;
}

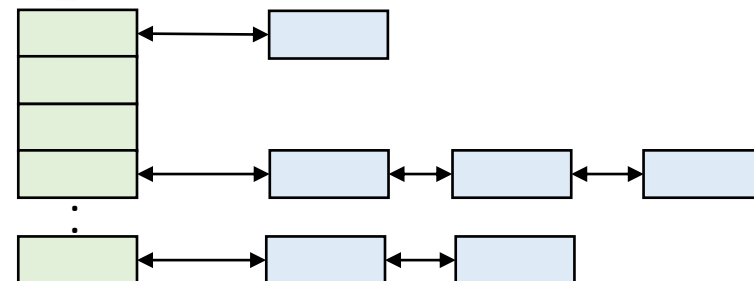
struct dentry * __d_lookup(struct dentry * parent, struct qstr * name)
{
    unsigned int len = name->len;
    unsigned int hash = name->hash;
    const unsigned char *str = name->name;
    struct list_head *head = d_hash(parent, hash);
    struct list_head *tmp;

    tmp = head->next;
    for (;;) {
        struct dentry * dentry = list_entry(tmp, struct dentry, d_hash);
        if (tmp == head)
            break;
        tmp = tmp->next;
        if (dentry->d_name.hash != hash)
            continue;
        if (dentry->d_parent != parent)
            continue;
        if (parent->d_op && parent->d_op->d_compare) {
            if (parent->d_op->d_compare(parent, &dentry->d_name, name))
                continue;
        }
        else {
            if (dentry->d_name.len != len)
                continue;
            if (memcmp(dentry->d_name.name, str, len))
                continue;
        }
        return dentry;
    }
    return NULL;
}
```

Source code from: v2.5.61

- v2.5.61 and earlier
- dcache_lock
 - ✓ protect the hash chain (hash table), d_child, d_alias, d_lru lists and d_inode

dentry_hashtable



Case Study: Code locking: dentry lookup

```
struct dentry * d_lookup(struct dentry * parent, struct qstr * name)
{
    struct dentry * dentry;
    spin_lock(&dcache_lock);
    dentry = __d_lookup(parent, name);
    if (dentry)
        __dget_locked(dentry);
    spin_unlock(&dcache_lock);
    return dentry;
}

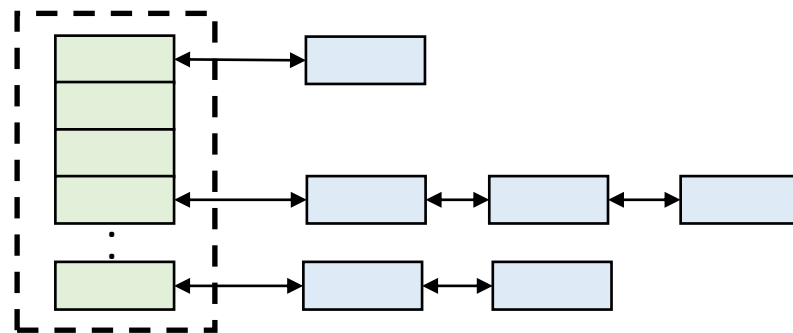
struct dentry * __d_lookup(struct dentry * parent, struct qstr * name)
{
    unsigned int len = name->len;
    unsigned int hash = name->hash;
    const unsigned char *str = name->name;
    struct list_head *head = d_hash(parent, hash);
    struct list_head *tmp;

    tmp = head->next;
    for (;;) {
        struct dentry * dentry = list_entry(tmp, struct dentry, d_hash);
        if (tmp == head)
            break;
        tmp = tmp->next;
        if (dentry->d_name.hash != hash)
            continue;
        if (dentry->d_parent != parent)
            continue;
        if (parent->d_op && parent->d_op->d_compare) {
            if (parent->d_op->d_compare(parent, &dentry->d_name, name))
                continue;
        }
        if (dentry->d_name.len != len)
            continue;
        if (memcmp(dentry->d_name.name, str, len))
            continue;
        return dentry;
    }
    return NULL;
}
```

Code locking

- v2.5.61 and earlier
- dcache_lock
 - ✓ protect the hash chain (hash table), d_child, d_alias, d_lru lists and d_inode

dentry_hashtable



Code locking by `dcache\_lock`

- dcache_lock protects the whole hash table
- More lock contention, poor scalability

Case Study: Data locking: dentry lookup

```
struct dentry * d_lookup(struct dentry * parent, struct qstr * name)
{
    unsigned int len = name->len;
    unsigned int hash = name->hash;
    const unsigned char *str = name->name;
    struct list_head *head = d_hash(parent, hash);
    struct dentry *found = NULL;
    struct list_head *tmp;
    int lookup_count = 0;

    rcu_read_lock();

    /* lookup is terminated when flow reaches any bucket head */
    for(tmp = head->next; !is_bucket(tmp); tmp = tmp->next) {
        struct dentry *dentry;
        unsigned long move_count;
        struct qstr * qstr;

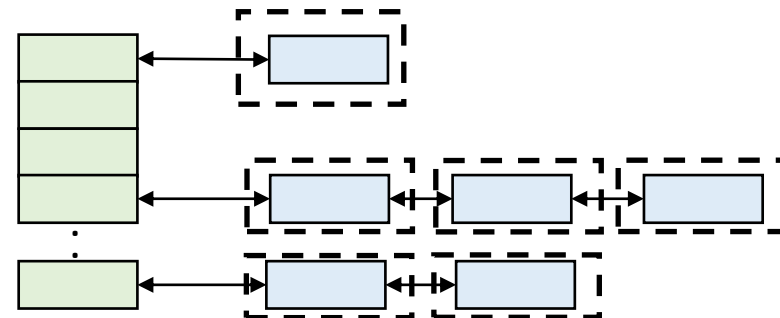
+--- 29 lines: smp_read_barrier_depends();-----
        if (parent->d_op && parent->d_op->d_compare) {
            if (parent->d_op->d_compare(parent, qstr, name))
                continue;
        } else {
            if (qstr->len != len)
                continue;
            if (memcmp(qstr->name, str, len))
                continue;
        }
        spin_lock(&dentry->d_lock); ← Data locking
        /*
         * If dentry is moved, fail the lookup
         */
        if (unlikely(move_count != dentry->d_move_count))
            break;
        if (!d_unhashed(dentry))
            found = dget(dentry);
        spin_unlock(&dentry->d_lock);
        break;
    }
    rcu_read_unlock();

    return found;
}
```

Source code from: v2.5.62

- v2.5.62 and later
- RCU
 - ✓ lock-free for dcache (dentry) look-up
 - ✓ No need to acquire `dcache\_lock` when traversing the hash table in d_lookup
 - ✓ Rely on RCU to ensure the dentry has not been *freed*.
- dcache_lock
 - ✓ dcache_lock must be taken for the followings:
 - ✓ Traversing and updating
 - ✓ Hashtable update

dentry_hashtable



Data locking by `dentry->d\_lock`

- Improve lock contention
- Better scalability

Case Study: Data locking: dentry lookup - seqlock

commit 1b8910cf1042f2de2cdd1dc26e4fb677b6dad7ed

Author: Andrew Morton <akpm@digeo.com>

Date: Wed Apr 2 16:28:49 2003 -0800

[PATCH] real_lookup race fix

From: Maneesh Soni <maneesh@in.ibm.com>

Here is a patch to use seqlock for real_lookup race with d_lookup as suggested by Linus. The race condition can result in duplicate dentry when d_lookup fails due concurrent d_move in some unrelated directory.

Apart from real_lookup, lookup_hash()->cached_lookup() can also fail due to same reason. So, for that I am doing the d_lookup again.

Now we have __d_lookup (called from do_lookup() during pathwalk) and d_lookup which uses seqlock to protect against rename race.

dcachebench numbers (lower is better) don't have much difference on a 4-way PIII xeon SMP box.

```
struct dentry * d_lookup(struct dentry * parent, struct qstr * name)
{
    struct dentry * dentry = NULL;
    unsigned long seq;

    do {
        seq = read_seqbegin(&rename_lock);
        dentry = __d_lookup(parent, name);
        if (dentry)
            break;
    } while (read_seqretry(&rename_lock, seq));
    return dentry;
}
```

Source code from: v2.5.67

```
struct dentry * __d_lookup(struct dentry * parent, struct qstr * name)
{
    unsigned int len = name->len;
    unsigned int hash = name->hash;
    const unsigned char *str = name->name;
    struct hlist_head *head = d_hash(parent, hash);
    struct dentry *found = NULL;
    struct hlist_node *node;

    rcu_read_lock();

    hlist_for_each (node, head) {
        struct dentry *dentry;
        unsigned long move_count;
        struct qstr *qstr;

        prefetch(node->next);

        smp_read_barrier_depends();
        dentry = hlist_entry(node, struct dentry, d_hash);

+-- 29 lines: if lookup ends up in a different bucket -----
        spin_lock(&dentry->d_lock);
        /*
         * If dentry is moved, fail the lookup
         */
        if (likely(move_count == dentry->d_move_count)) {
            if (!d_unhashed(dentry)) {
                atomic_inc(&dentry->d_count);
                dentry->d_vfs_flags |= DCACHE_REFERENCED;
                found = dentry;
            }
        }
        spin_unlock(&dentry->d_lock);
        break;
    }
    rcu_read_unlock();

    return found;
}
```


Case Study: Data locking: dentry lookup - seqlock

commit 1b8910cf1042f2de2cdd1dc26e4fb677b6dad7ed

Author: Andrew Morton <akpm@digeo.com>

Date: Wed Apr 2 16:28:49 2003 -0800

[PATCH] real_lookup race fix

From: Maneesh Soni <maneesh@in.ibm.com>

Here is a patch to use seqlock for real_lookup race with d_lookup as suggested by Linus. The race condition can result in duplicate dentry when d_lookup fails due concurrent d_move in some unrelated directory.

Apart from real_lookup, lookup_hash()->cached_lookup() can also fail due to same reason. So, for that I am doing the d_lookup again.

Now we have __d_lookup (called from do_lookup() during pathwalk) and d_lookup which uses seqlock to protect against rename race.

dcachebench numbers (lower is better) don't have much difference on a 4-way PIII xeon SMP box.

```
struct dentry * d_lookup(struct dentry * parent, struct qstr * name)
{
    struct dentry * dentry = NULL;
    unsigned long seq;

    do {
        seq = read_seqbegin(&rename_lock);
        dentry = __d_lookup(parent, name);
        if (dentry)
            break;
    } while (read_seqretry(&rename_lock, seq));
    return dentry;
}
```

Source code from: v2.5.67

```
struct dentry * __d_lookup(struct dentry * parent, struct qstr * name)
{
    unsigned int len = name->len;
    unsigned int hash = name->hash;
    const unsigned char *str = name->name;
    struct hlist_head *head = d_hash(parent, hash);
    struct dentry *found = NULL;
    struct hlist_node *node;
```

```
    rcu_read_lock();
```

```
    hlist_for_each (node, head) {
        struct dentry *dentry;
        unsigned long move_count;
        struct qstr *qstr;
```

```
        prefetch(node->next);
```

```
        smp_read_barrier_depends();
        dentry = hlist_entry(node, struct dentry, d_hash);
```

+-- 29 lines: if lookup ends up in a different bucket -----

```
        spin_lock(&dentry->d_lock);
```

```
        /*
         * If dentry is moved, fail the lookup
         */
```

```
        if (likely(move_count == dentry->d_move_count)) {
            if (!d_unhashed(dentry)) {
                atomic_inc(&dentry->d_count);
                dentry->d_vfs_flags |= DCACHE_REFERENCED;
                found = dentry;
```

```
            }
        }
        spin_unlock(&dentry->d_lock);
        break;
```

```
    }
    rcu_read_unlock();
```

This approach is still used in latest kernel (v6.3)

Reference

- [Is Parallel Programming Hard, And, If So, What Can You Do About It?](#)
- [What is RCU? – “Read, Copy, Update”](#)
- [What Does It Mean To Be An RCU Implementation?](#)
- <https://docs.kernel.org/RCU/index.html>
- [Using RCU for linked lists — a case study](#)
- [Scaling dcache with RCU](#)
- Sleepable RCU (SRCU)
 - <https://lwn.net/Articles/253651/>
 - <https://zhuanlan.zhihu.com/p/90223380>
- Preemptible RCU
 - <https://lwn.net/Articles/253651/>
 - <https://zhuanlan.zhihu.com/p/90223380>

Backup

RCU (Read-Copy-Update)

- Non-blocking synchronization

- ✓ Deadlock Immunity: RCU read-side primitives do not block, spin or even do backwards branches → execution time is deterministic.

- Exception (programming error):

```
rcu_read_lock();  
synchronize_rcu();  
rcu_read_unlock();
```

- Immunity to priority inversion

- Low-priority RCU readers cannot prevent a high-priority RCU updater from acquiring the update-side lock

- A low-priority RCU updater cannot prevent high-priority RCU readers from entering read-side critical section

- [-rt kernel] RCU is susceptible to priority inversion scenarios:

- A High-priority process blocked waiting for an RCU grace period to elapse can be blocked by low-priority RCU readers. --> Solve by RCU priority boosting.

- [RCU priority boosting] Require rcu_read_unlock() do deboosting, which entails acquiring scheduler locks.

- Need to avoid deadlocks within the scheduler and RCU: v5.15 kernel requires RCU to avoid invoking the scheduler while holding any of RCU's locks

- rcu_read_unlock() is not always lockless when RCU priority boosting is enabled.

RCU Properties

- Reader

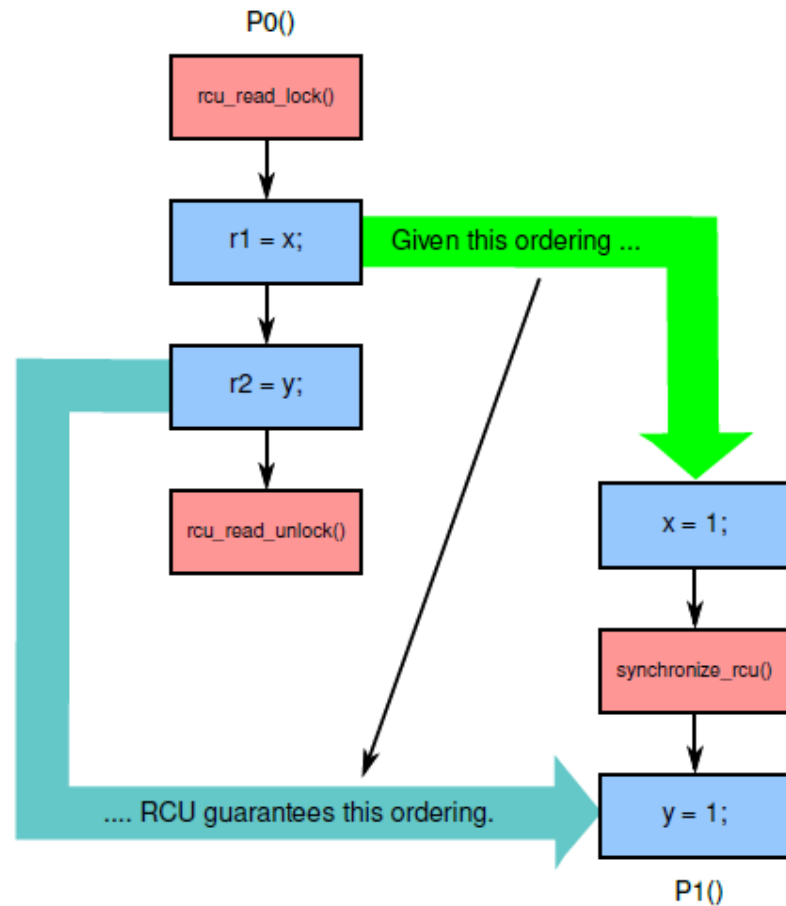
- ✓ Reads need not wait for updates
 - Low-cost or even no-cost readers → excellent scalability
- ✓ Each reader has a coherent view of each object within the block of `rcu_read_lock()` and `rcu_read_unlock()`

- Updater

- ✓ `synchronized_rcu()`: ensure that objects are not freed until after the completion of all readers that might be using them.
- ✓ `rcu_assign_pointer()` and `rcu_dereference()`: efficient and scalable mechanisms for publishing and reading new versions of an object.

Wait for pre-existing RCU readers

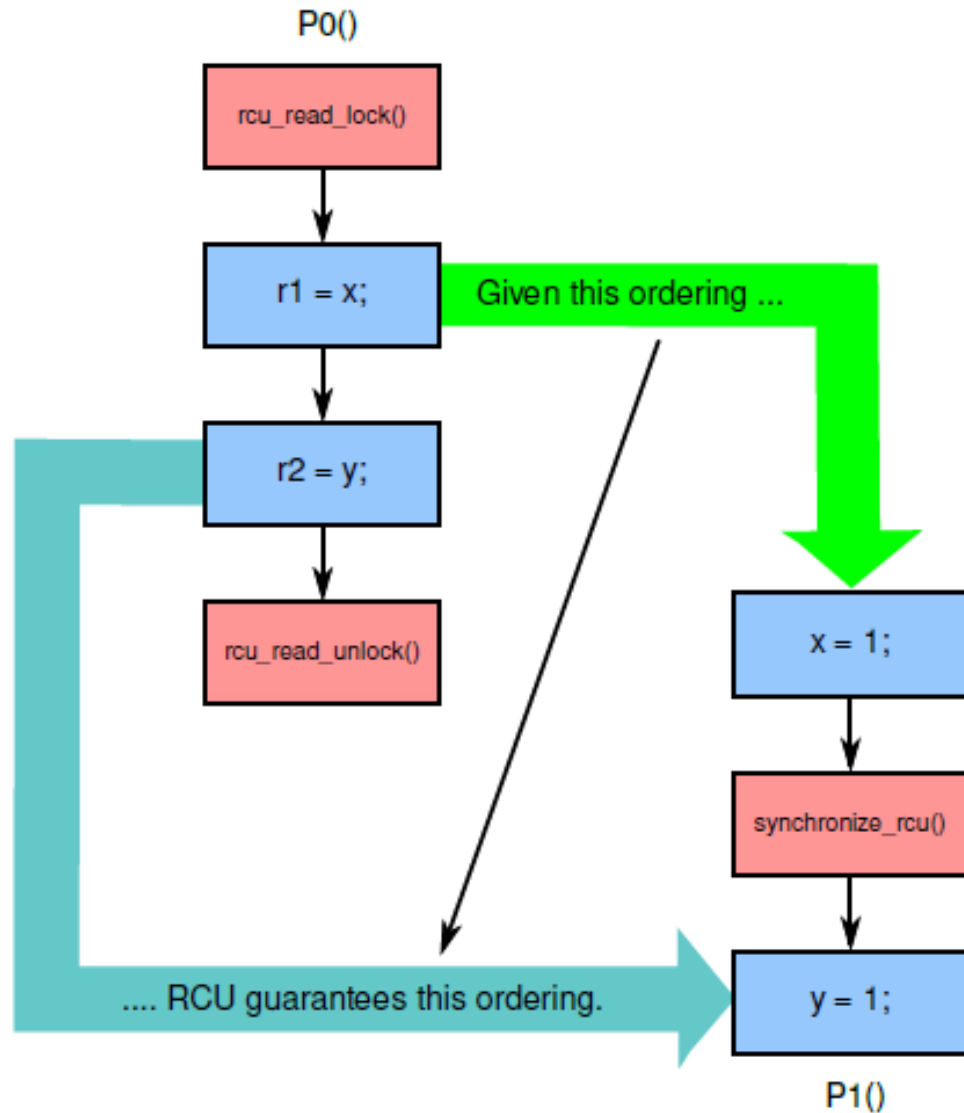
- Wait for the RCU read-side critical section: `rcu_read_lock()/rcu_read_unlock()`
 - ✓ Illegal to sleep within an RCU read-side critical section because a context switch is a quiescent state



* If any portion of a given critical section precedes the beginning of a given grace period, then RCU guarantees that all of that critical section will precede the end of that grace period.

Figure 9.11: RCU Reader and Later Grace Period

Wait for pre-existing RCU readers: if-then relationship



- If any portion of a given critical section precedes the beginning of a given grace period, then RCU guarantees that all of that critical section will precede the end of that grace period.
✓ $r1 = 0$ and $r2 = 0$

Figure 9.11: RCU Reader and Later Grace Period

Wait for pre-existing RCU readers: if-then relationship

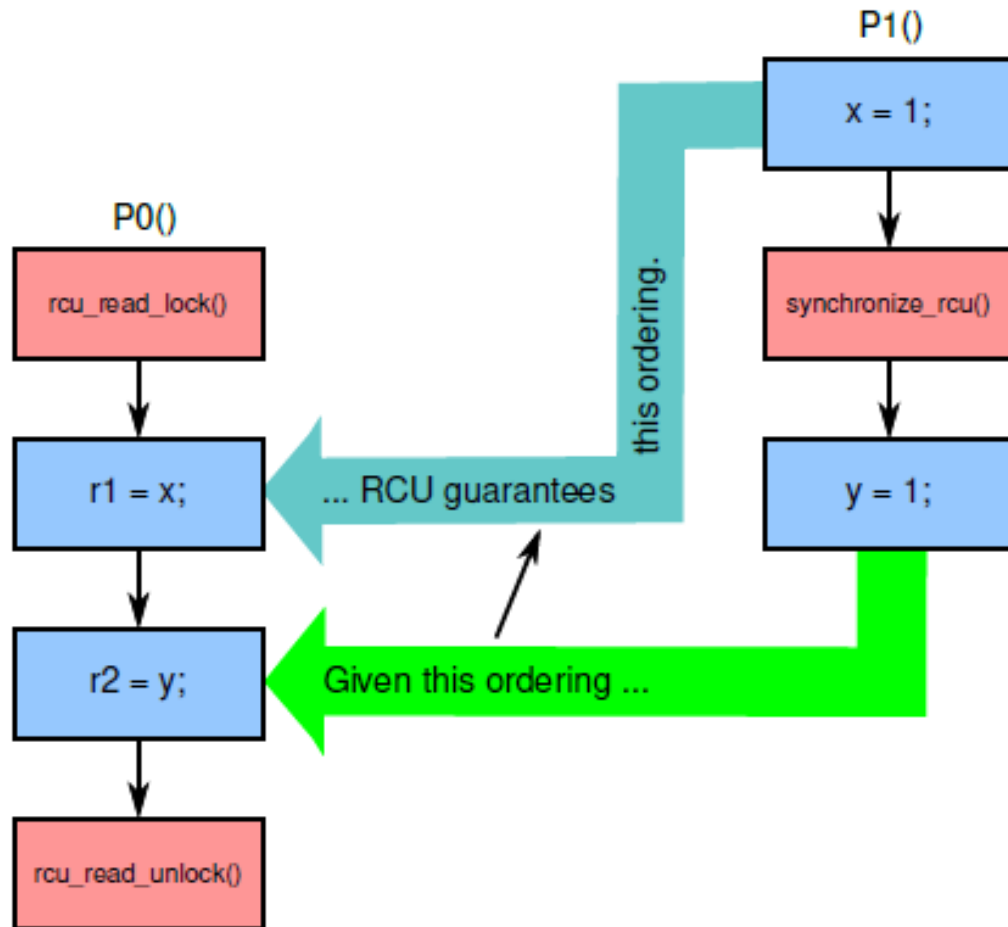
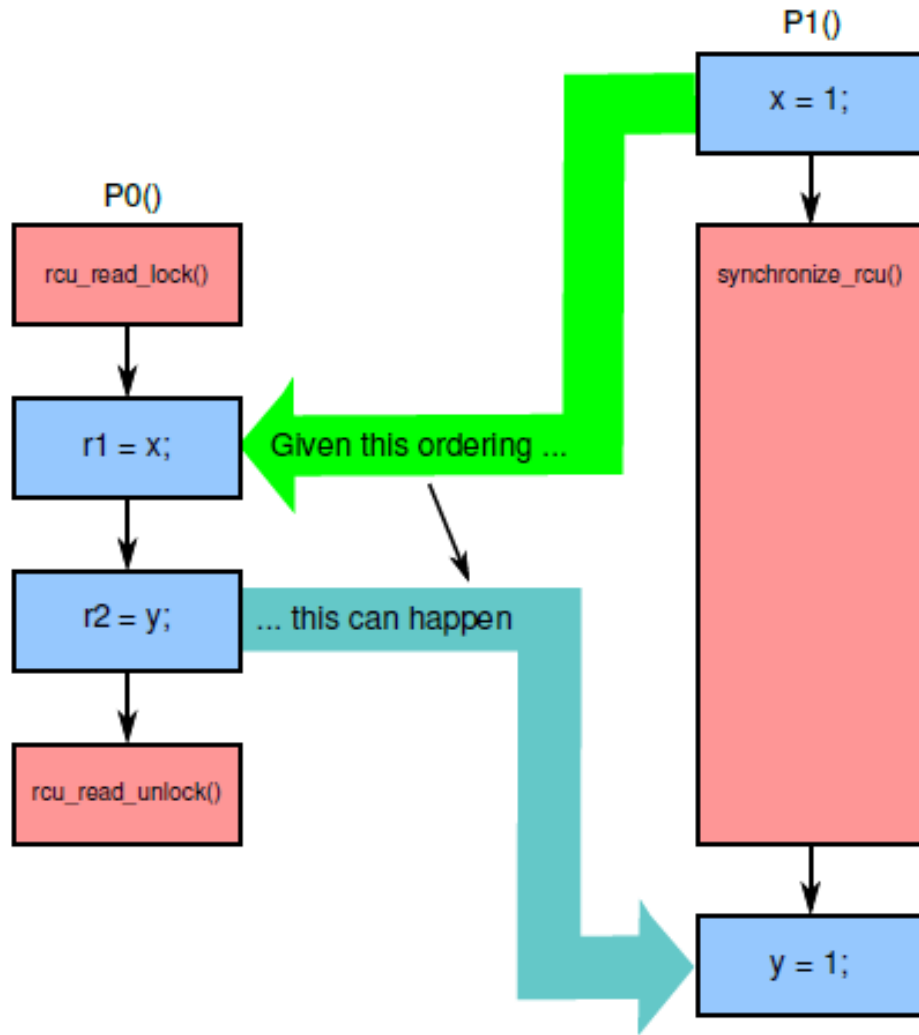


Figure 9.12: RCU Reader and Earlier Grace Period

- If any portion of a given critical section follows the end of a given grace period, then RCU guarantees that all of that critical section will follow the beginning of that grace period.
 - ✓ $r1 = 1$ and $r2 = 1$
- What would happen if the order of P()'s two accesses was reversed?
 - ✓ Nothing change because loads from x and y are in the same RCU read-side critical section.

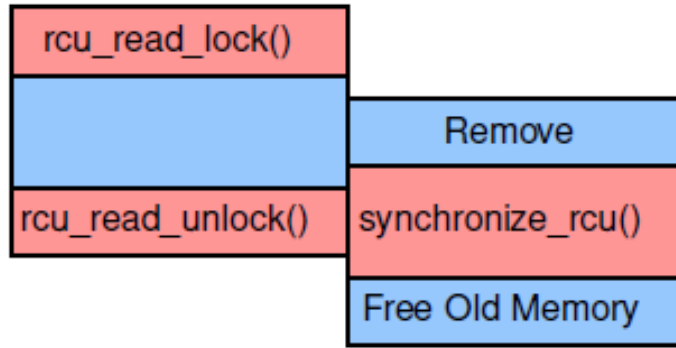
Wait for pre-existing RCU readers: if-then relationship



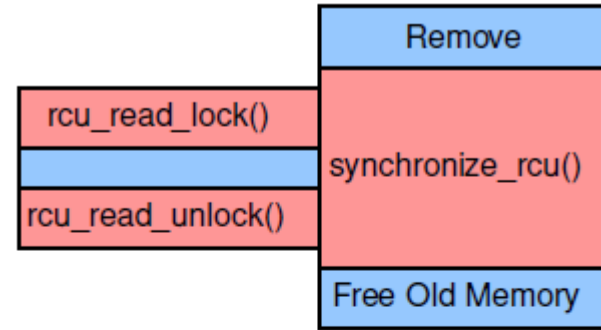
- An RCU read-side critical section can be completely overlapped by an RCU grace period.
 - ✓ `r1 = 1` and `r2 = 0`
 - ✓ Cannot be `r1 = 0` and `r2 = 1`

Figure 9.13: RCU Reader Within Grace Period

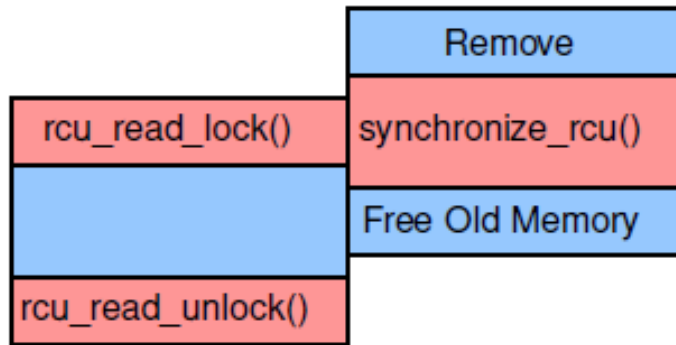
RCU Grace-Period Ordering Guarantee



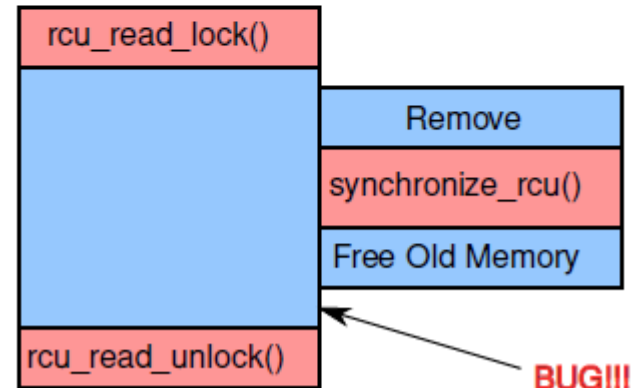
(1) Reader precedes removal



(3) Reader within grace period



(2) Removal precedes reader



(4) Grace period within reader (BUG!!!)

Given a grace period, each readers ends before the end of that grace period, starts after the beginning of that grace period, or both.

Maintain Multiple Versions of Recently Updated Objects

- RCU accommodates synchronization-free readers (weak temporal synchronization) by maintaining multiple versions of data

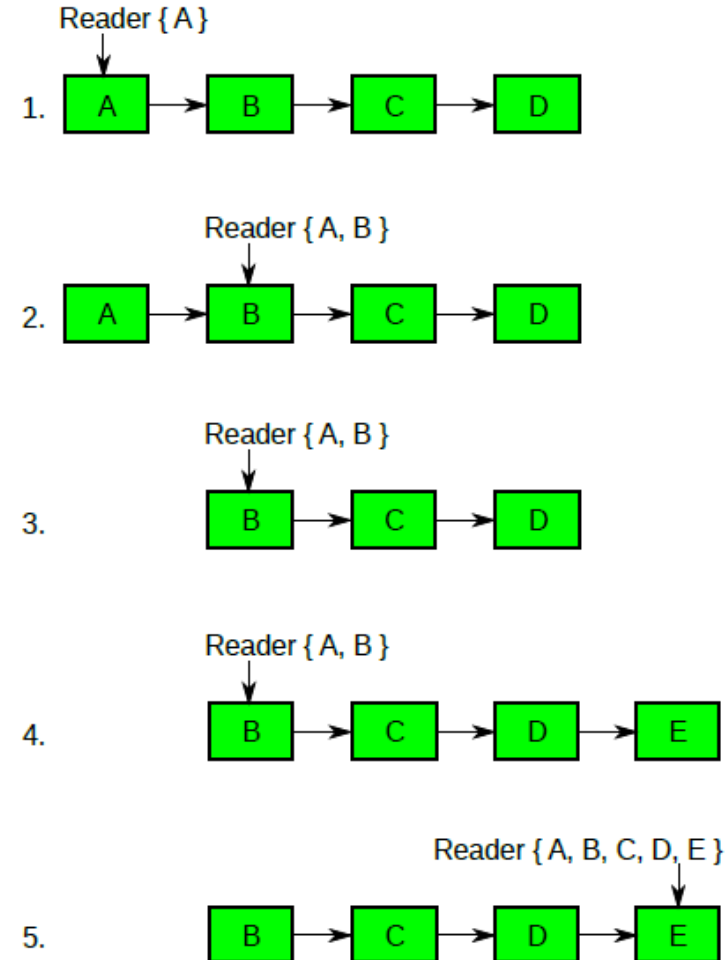


Figure 9.15: Multiple RCU Data-Structure Versions

Design Principle: Avoid Expensive Operations

Need to be here!

(Partitioning/RCU/hazptr)

But can't always!

16-CPU 2.8GHz Intel X5550 (Nehalem) System

Operation	Cost (ns)	Ratio
Clock period	0.4	1
"Best-case" CAS	12.2	33.8
Best-case lock	25.6	71.2
Single cache miss	12.9	35.8
CAS cache miss	7.0	19.4
Single cache miss (off-core)	31.2	86.6
CAS cache miss (off-core)	31.2	86.5
Single cache miss (off-socket)	92.4	256.7
CAS cache miss (off-socket)	95.9	266.4

Heavily
optimized
reader-writer
lock might get
here for readers
(but too bad
about those
poor writers...)

Typical synchronization
mechanisms do this a lot

Core RCU API: Temporal vs. Spatial

- `rcu_read_lock()`: Begin reader
- `rcu_read_unlock()`: End reader
- `synchronize_rcu()`: Wait for pre-existing readers
- `call_rcu()`: Invoke function after pre-existing readers complete
- `rcu_dereference()`: Load RCU-protected pointer
- `rcu_dereference_protected()`: Ditto, but update-side locked
- `rcu_assign_pointer()`: Update RCU-protected pointer
 - With `xchg()` standing in for these last two

Case study: Access task_struct linked list

```
__cacheline_aligned DEFINE_RWLOCK(tasklist_lock);
```

